Compte Rendu Complete de TP Q&T

Made by:
Ahmed R. Hedhili

Supervised by:
Dr. Amen Bidani

A.U 2025/2026

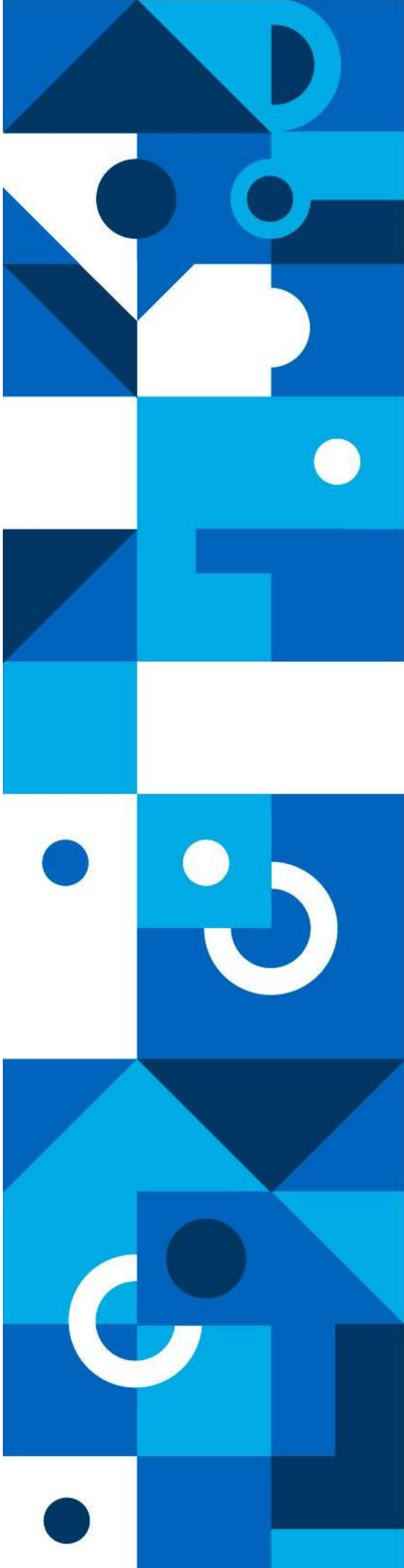
Acknowledgments

I would like to express my sincere gratitude to ***Dr. Amen Bidani*** *for her invaluable guidance and support throughout this TP. Her expertise has significantly enhanced my understanding of software testing systems and their practical implementation.*

Table of Contents

TP1: Unit Testing in Java using JUnit 5 4
TP2: Test Driven Development in Java 13
TP3: Automated testing with JUnit and Spring..... 21
TP4: Integration tests and BDD methodology 30
TP5: ChatGPT to facilitate software test automation..... 38
TP++: Testing using Python and PyTest 46

TP1: Unit Testing in Java using JUnit 5



Introduction

Unit testing is a fundamental pillar in the software development lifecycle, aiming to isolate specific parts of code and verify their correct and independent functionality. This part of report focuses on how to set up and execute these tests in a Java environment using the **JUnit 5** framework, based on the educational content provided in the attached document.

Core Concepts of Unit Testing

The basic process of testing a software unit involves the following steps:

1. **Instantiation**: Create an object of the class to be tested.
2. **Initialization**: Set up the initial state of the object.
3. **Execution**: Invoke the method to be tested with the necessary parameters.
4. **Verification**: Compare the actual result with the expected result using assertions.

The JUnit 5 Framework

JUnit is an open-source and widely used framework for automating unit tests in Java. This framework ensures that the code still meets software requirements even after modifications or improvements, which is known as "**non-regression**" testing.

Annotations in JUnit 5

Annotations are used to guide the framework on how to handle test methods. The following table illustrates the most important annotations used:

Annotation	Purpose
@Test	Designates a method as a test method.
@BeforeEach	Executes the method before each test (ideal for setting up prerequisites).
@AfterEach	Executes the method after each test (ideal for cleanup operations).
@BeforeAll	Executes the method once before all tests (must be static).
@AfterAll	Executes the method once after all tests (must be static).
@Timeout	Specifies a maximum time for test execution; if exceeded, the test fails.

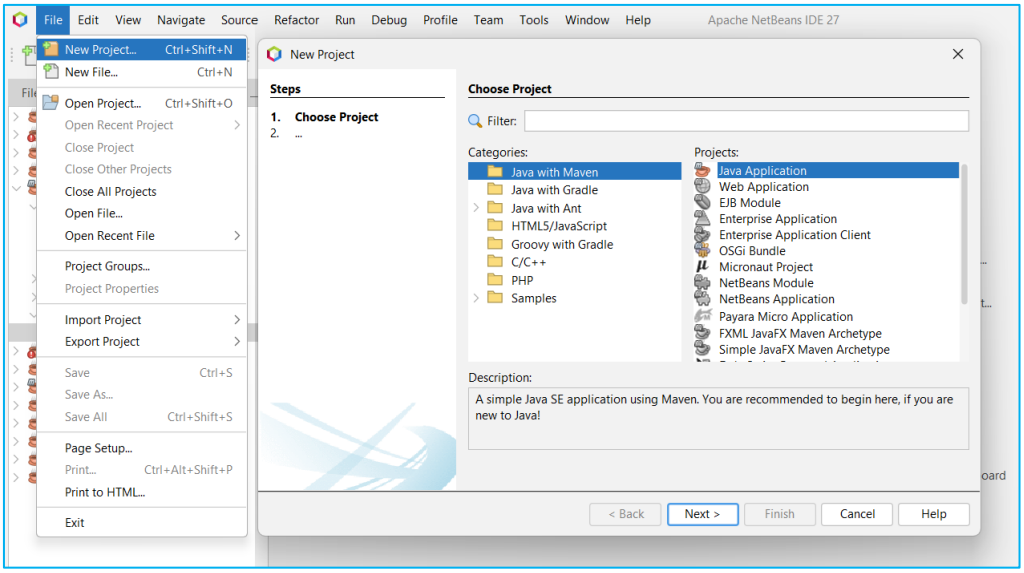
Practical Application using NetBeans Environment

Setting up a test project in **NetBeans** involves the following steps:

1. Project Creation:

Create a new project by selecting ****Java with Maven > Java Application****.

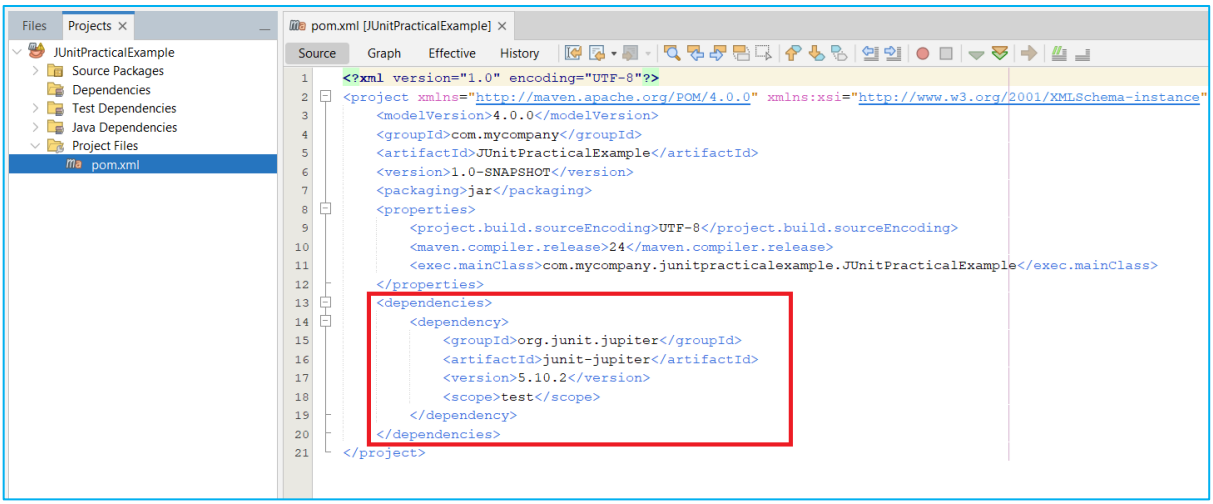
This ensures automated dependency management and a standardized project structure.



2. Configuring Dependencies:

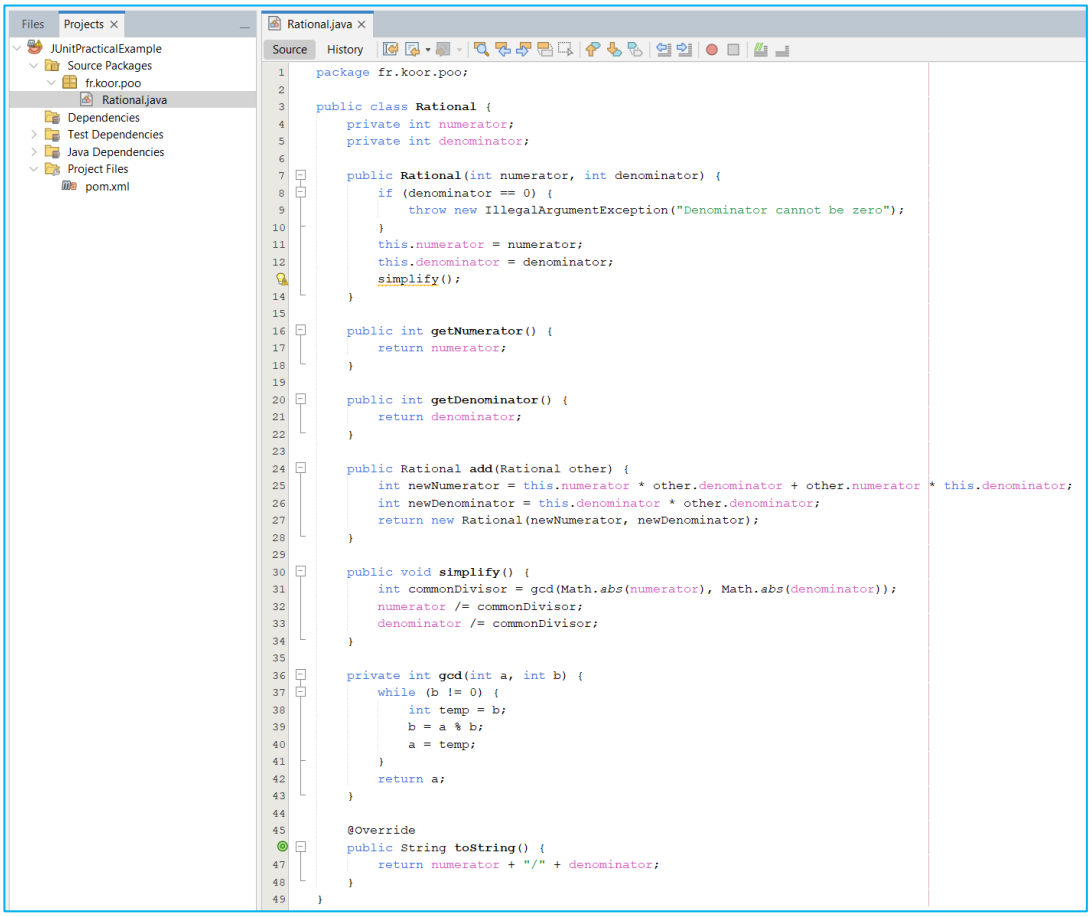
Since some **NetBeans** versions do not natively support **JUnit 5** in the default "Create Test" dropdown, the framework must be added manually.

This is achieved by editing the **pom.xml** file and inserting the **JUnit 5 (Jupiter)** dependency within the **<dependencies>** section. This step forces the IDE to fetch and integrate the modern testing library.



3. Source Code Implementation:

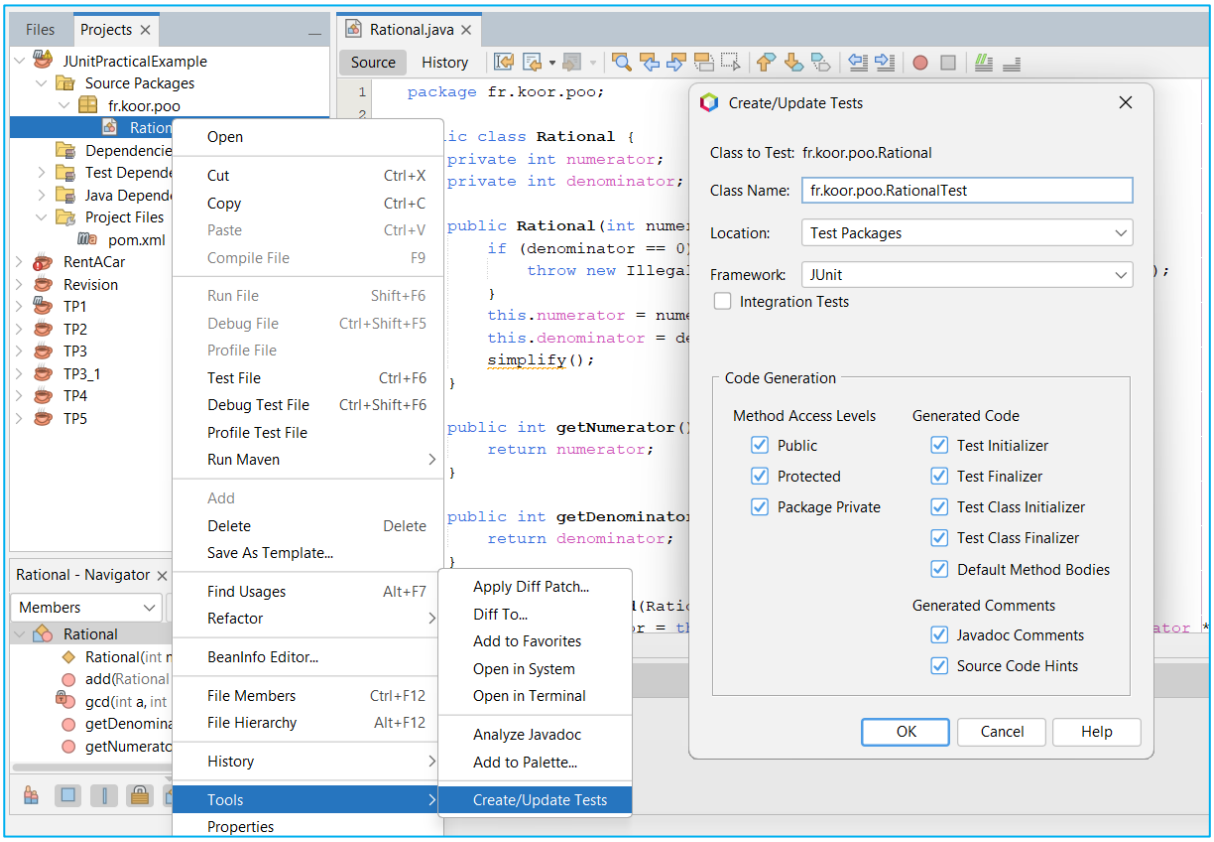
Create a package (e.g., **fr.koor.poo**) under **Source Packages** and implement the class to be tested, such as the **Rational** class, ensuring all business logic is correctly defined.



4. Generating Test Cases:

Right-click the class file and select **Tools > Create/Update Tests**.

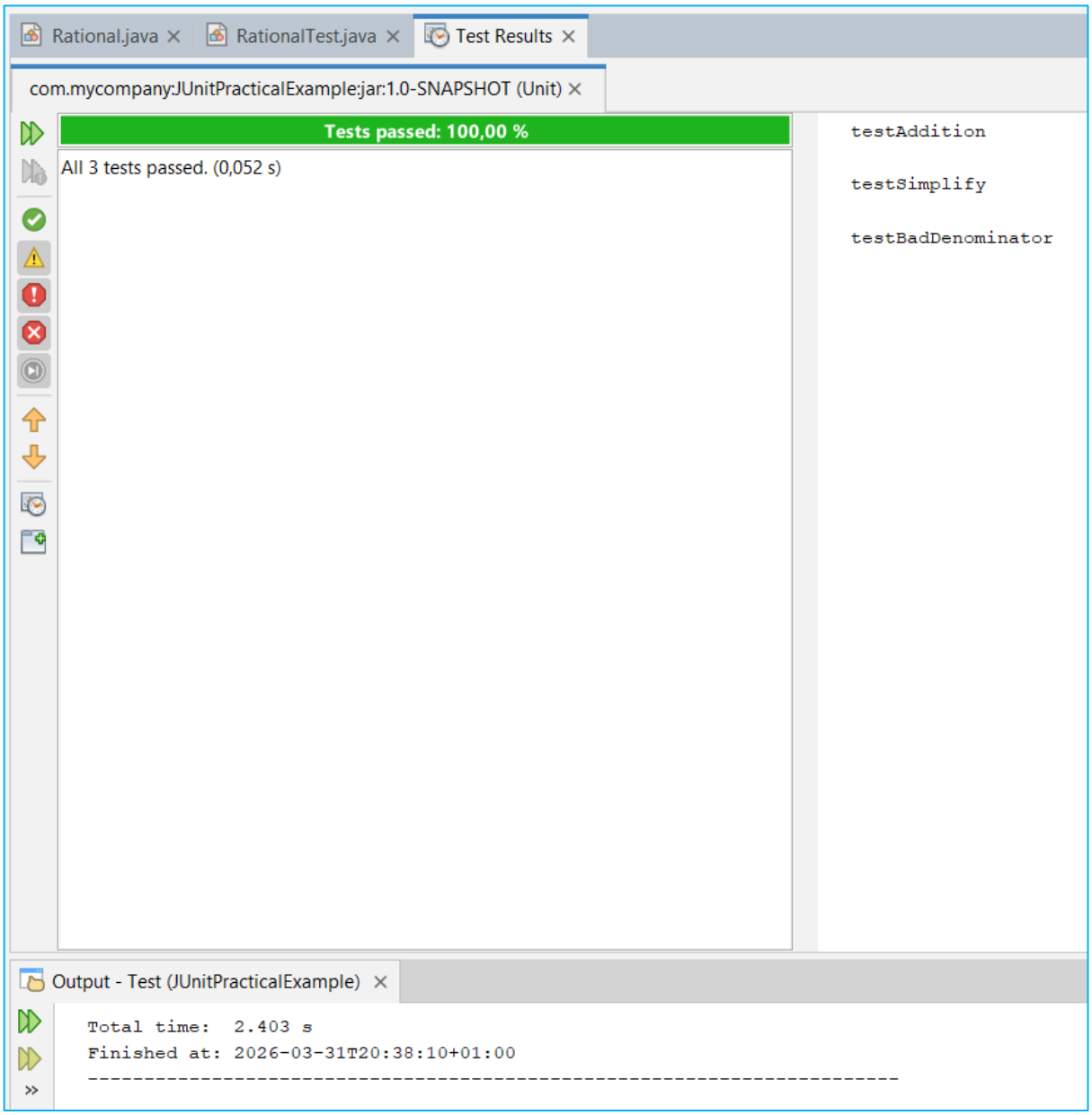
Even if you select "**JUnit**" from the limited dropdown, you can then manually update the generated test file to use **JUnit 5** imports (like **org.junit.jupiter.api.Test**) and modern features like **assertThrows**.


Files
Projects
JUnitPracticalExample
Source Packages
fr.koor.poo
Dependencies
Test Dependencies
Java Dependencies
Project Files
pom.xml
RentACar
Revision
TP1
TP2
TP3
TP3_1
TP4
TP5
Rational - Navigator
Members
Rational
Rational.java
Source
History
package fr.koor.poo;
Open
Cut Ctrl+X
Copy Ctrl+C
Paste Ctrl+V
Compile File F9
Run File Shift+F6
Debug File Ctrl+Shift+F5
Profile File
Test File Ctrl+F6
Debug Test File Ctrl+Shift+F6
Profile Test File
Run Maven
Add
Delete Delete
Save As Template...
Find Usages Alt+F7
Refactor
BeanInfo Editor...
File Members Ctrl+F12
File Hierarchy Alt+F12
History
Tools
Properties
Apply Diff Patch...
Diff To...
Add to Favorites
Open in System
Open in Terminal
Analyze Javadoc
Add to Palette...
Create/Update Tests
Create/Update Tests
Class to Test: fr.koor.poo.Rational
Class Name: fr.koor.poo.RationalTest
Location: Test Packages
Framework: JUnit
Integration Tests
Code Generation
Method Access Levels
Public
Protected
Package Private
Generated Code
Test Initializer
Test Finalizer
Test Class Initializer
Test Class Finalizer
Default Method Bodies
Generated Comments
Javadoc Comments
Source Code Hints
OK
Cancel
Help


Rational.java
RationalTest.java
Source
History
package fr.koor.poo;
import org.junit.jupiter.api.Test;
import static org.junit.jupiter.api.Assertions.*;
/**
*
* @author ahmed
*/
public class RationalTest {
public RationalTest() {
}
@Test
public void testAddition() {
System.out.println("testAddition");
Rational r1 = new Rational(1, 3);
Rational r2 = new Rational(2, 1);
Rational result = r1.add(r2);
assertEquals(7, result.getNumerator(), "Numerator should be 7");
assertEquals(3, result.getDenominator(), "Denominator should be 3");
}
@Test
public void testSimplify() {
System.out.println("testSimplify");
Rational r = new Rational(5005, 2431);
r.simplify();
assertEquals(35, r.getNumerator());
assertEquals(17, r.getDenominator());
}
@Test
public void testBadDenominator() {
System.out.println("testBadDenominator");
assertThrows(IllegalArgumentException.class, () -> {
new Rational(1, 0);
});
}
}

5. Execution and Verification:

Run the tests by pressing **Alt + F6**. The Test Results window will confirm the success, proving that the Manual **Maven** configuration successfully integrated JUnit 5 into the environment.

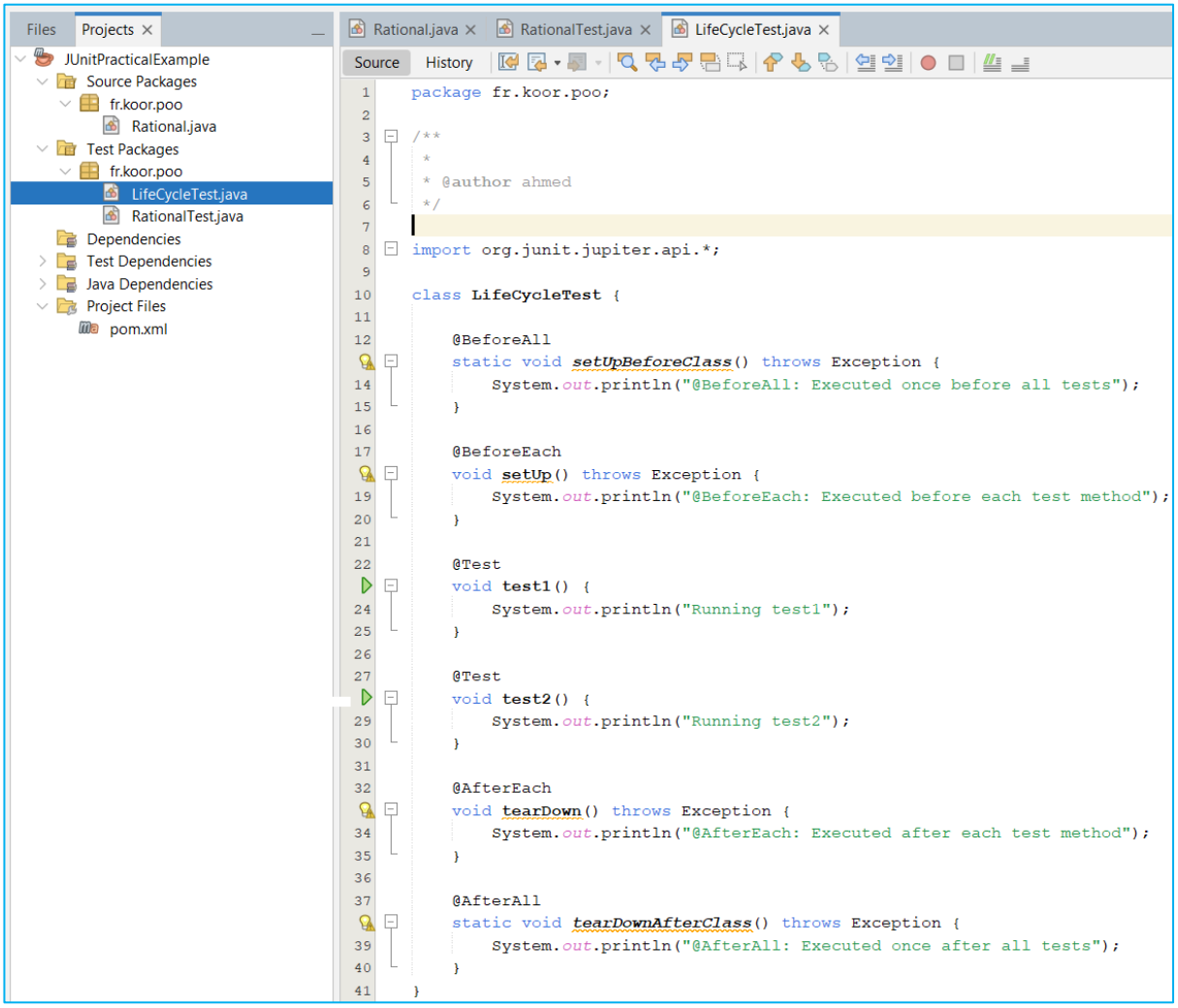


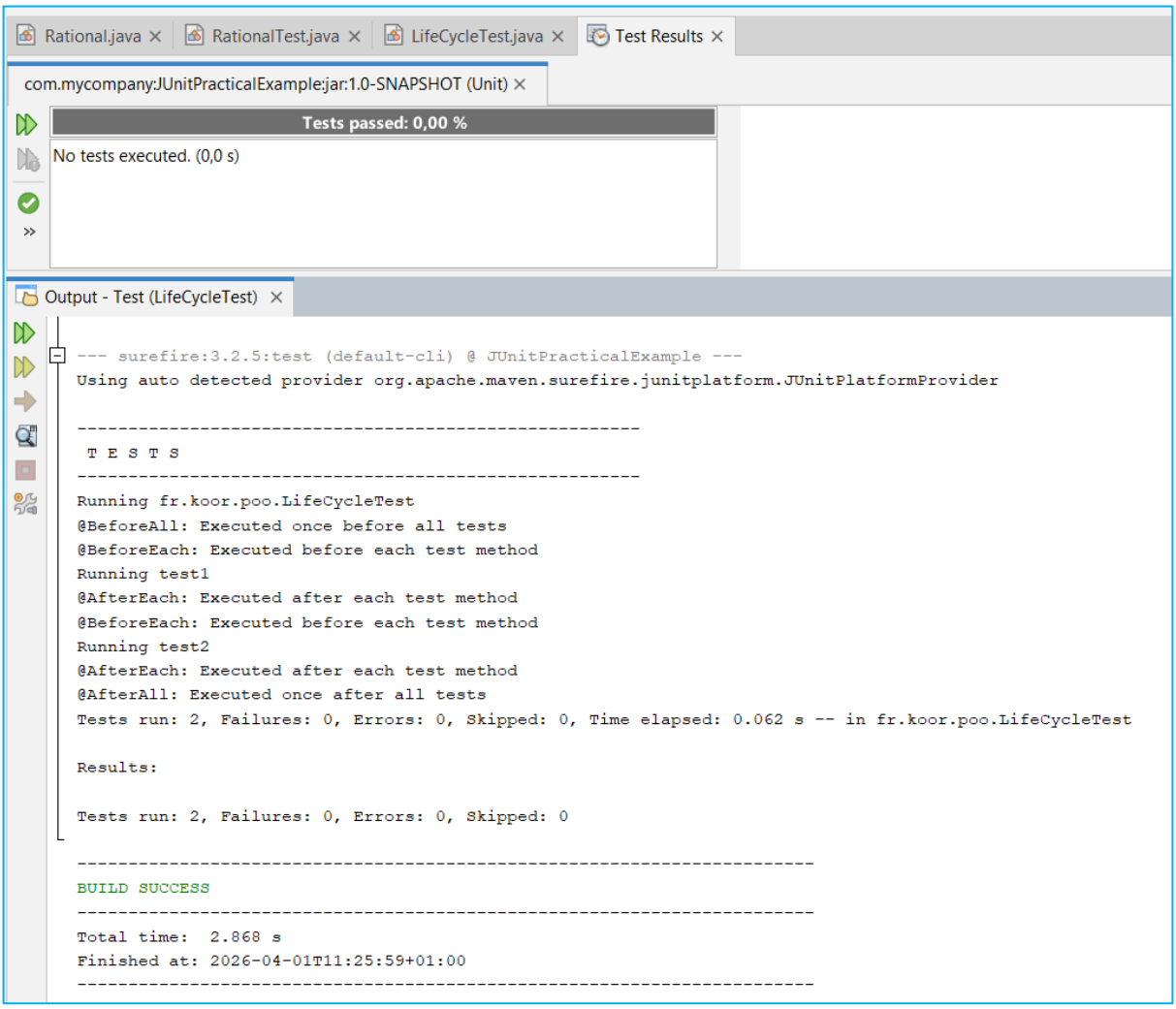
6. Test Lifecycle:

Test methods follow a specific chronological sequence:

- **@BeforeAll** is invoked once before all tests.

- For each **@Test** method: **@BeforeEach** is invoked, then the test is executed, then **@AfterEach** .
- **@AfterAll** is invoked once at the end.

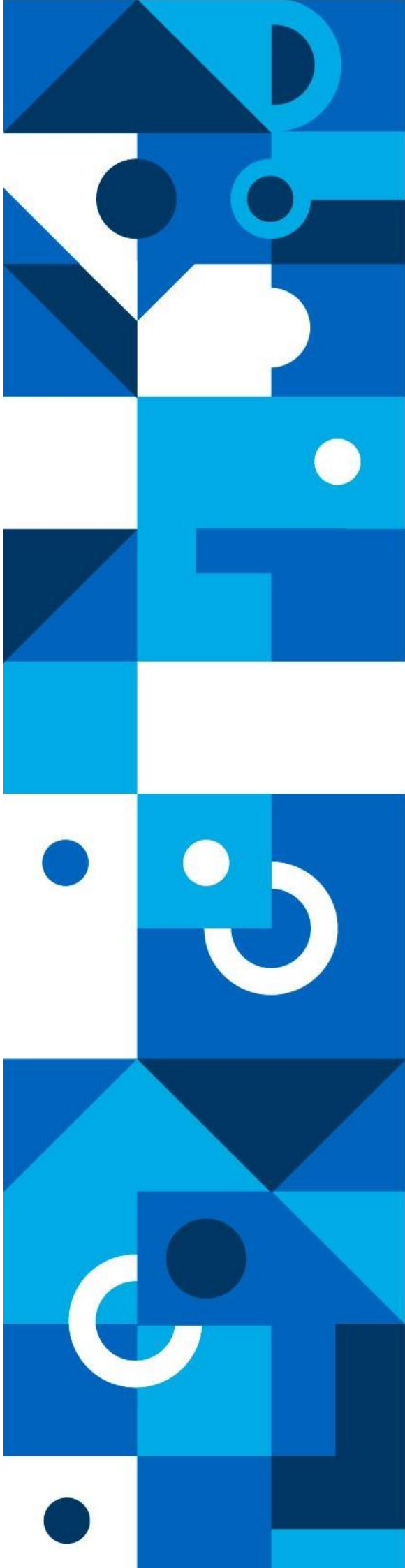




Conclusion

JUnit 5 is a powerful tool that directly contributes to improving the quality of software applications. By integrating automated tests into the development process, programmers can detect errors early and ensure code stability in the long term.

TP2: Test Driven Development in Java



Introduction to TDD (Test Driven Development)

Test Driven Development is a software development methodology based on a very short iteration cycle. Instead of writing code first and then testing it, we write the test first, then write the minimum amount of code to make that test pass and finally refactor the code.

The TDD Lifecycle (Red-Green-Refactor):

1. **Red Phase**: Write a test that fails (or doesn't even compile because the code doesn't exist yet).

2. **Green Phase**: Write the minimum amount of code to make the test pass.

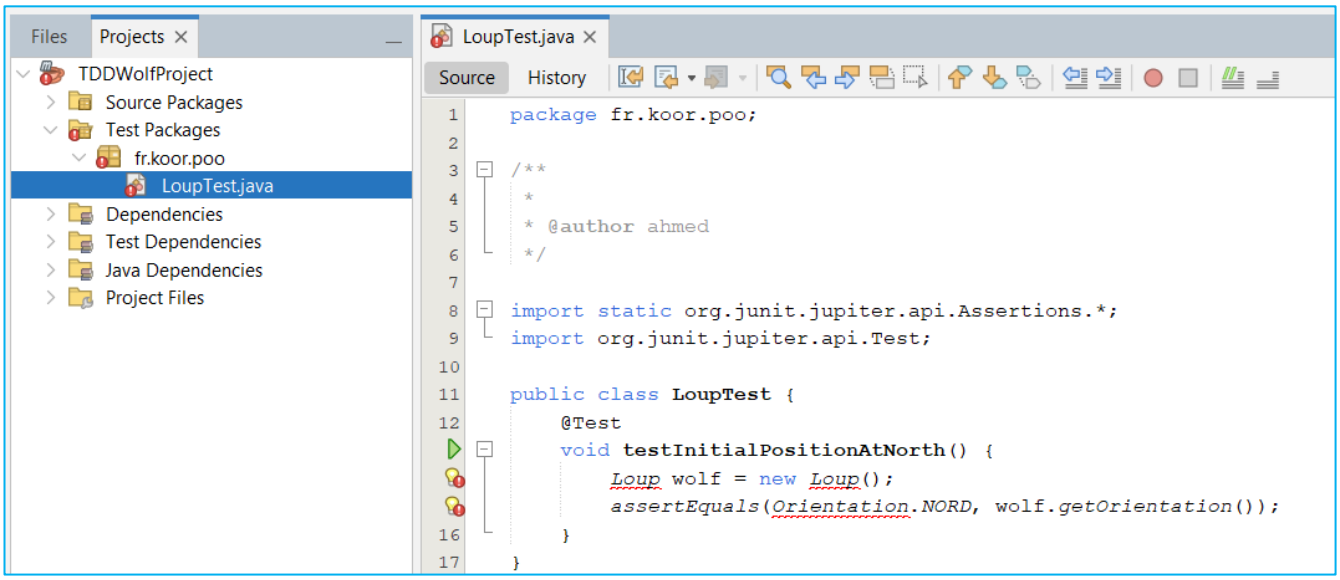
3. **Refactor Phase:** Improve and organize the code while ensuring that the tests remain successful.

Case Study: The Wolf Object (Loup)

The practical example in the document aims to create a "**Wolf**" object that can rotate clockwise (North, East, South, West), starting from the North position.

Phase 1: Creating the Project and Starting the Test (Red Phase):

1. **Create a new Maven project**: As we did previously, choose ****Java with Maven > Java Application**** and name it **TDDWolfProject**.
2. **Add the JUnit 5 library**: Don't forget to add the dependency in the **pom.xml** file as we learned in the first TP.
3. Go to Test Packages and create a new test class called **LoupTest**.

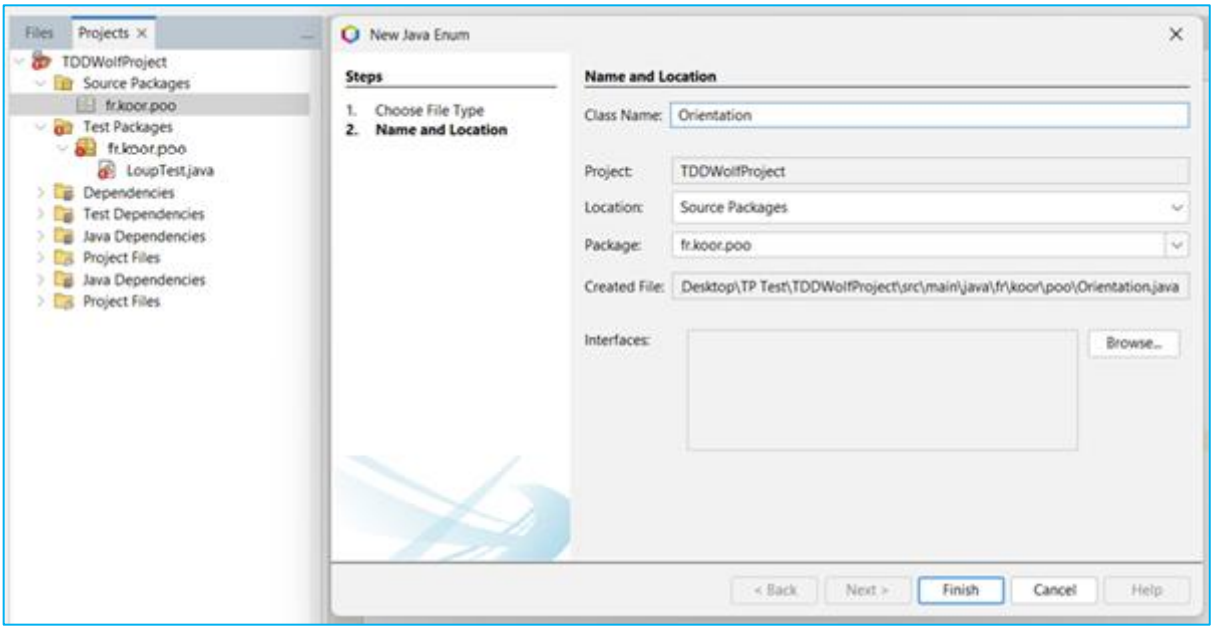


Observation: The code won't be compiled initially because the **Loup** and **Orientation** classes don't exist.

Phase 2: The Minimalist Solution (Green Phase):

Now, we will write as little code as possible just to make this test succeed

1. **Creating the Enum**: Create a new file named **Orientation**:



The Enum (short for Enumeration) is a special type in Java used to represent a fixed set of values (such as directions, days of the week, colors). In our example, we used it to ensure that the wolf's direction would not deviate from these four possibilities.

2. **Creating a Loup class**:

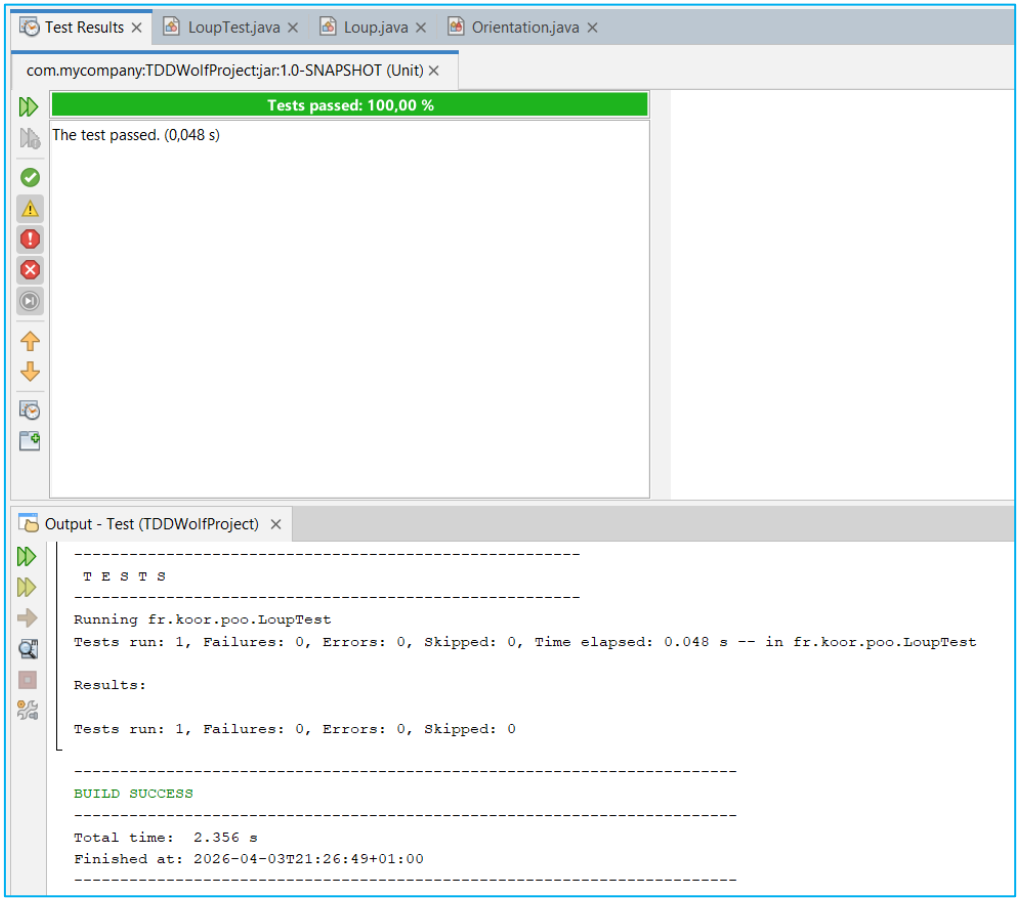
```java
package fr.koor.poo;

/**
 *
 * @author ahmed
 */

public class Loup {
    private Orientation orientation = Orientation.NORD;

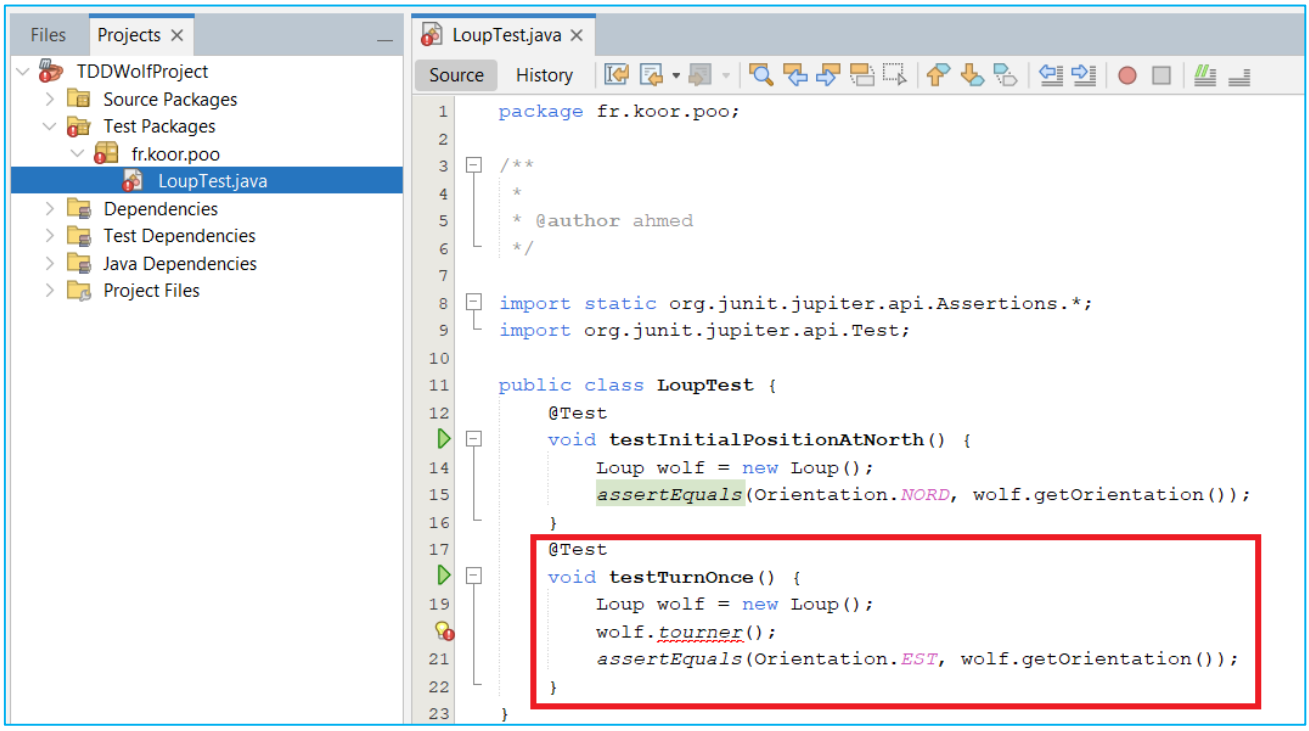
    public Orientation getOrientation() {
        return orientation;
    }
}
```

3. To operate: Press **Alt + F6**. You will get the green bar!



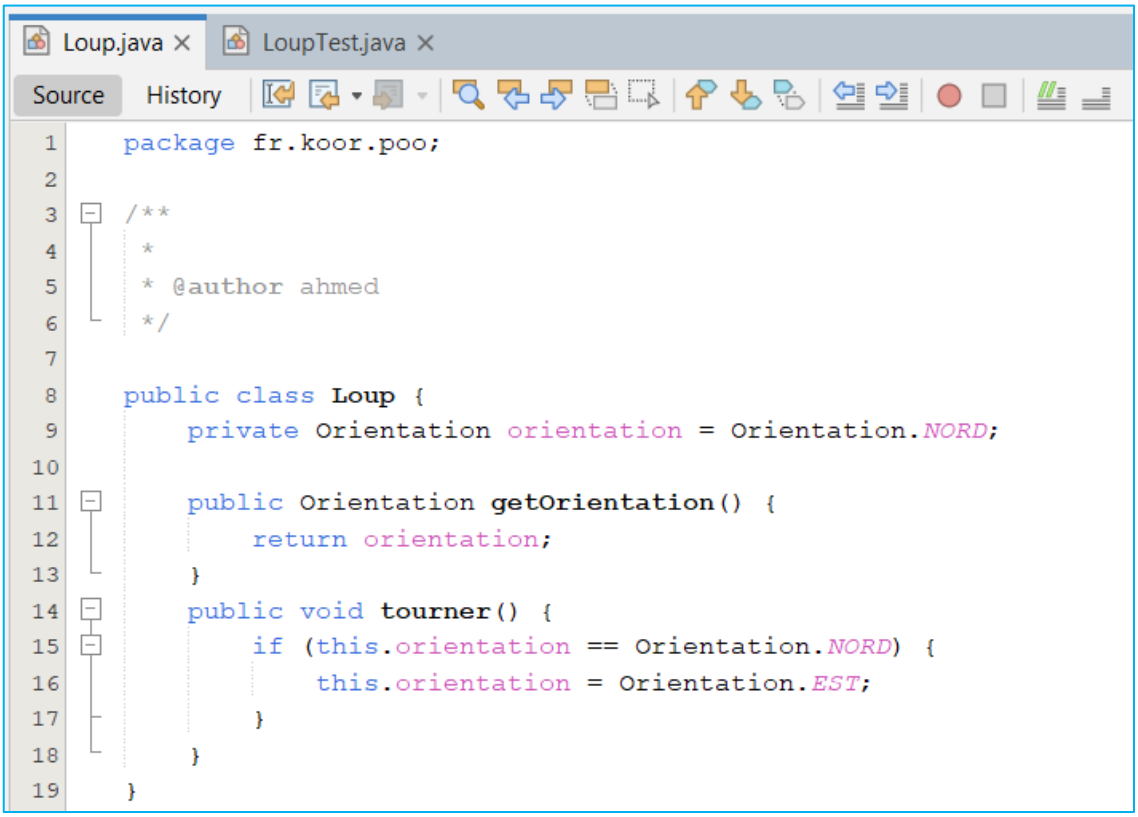
Phase 3: Adding the rotation feature (cycle repetition):

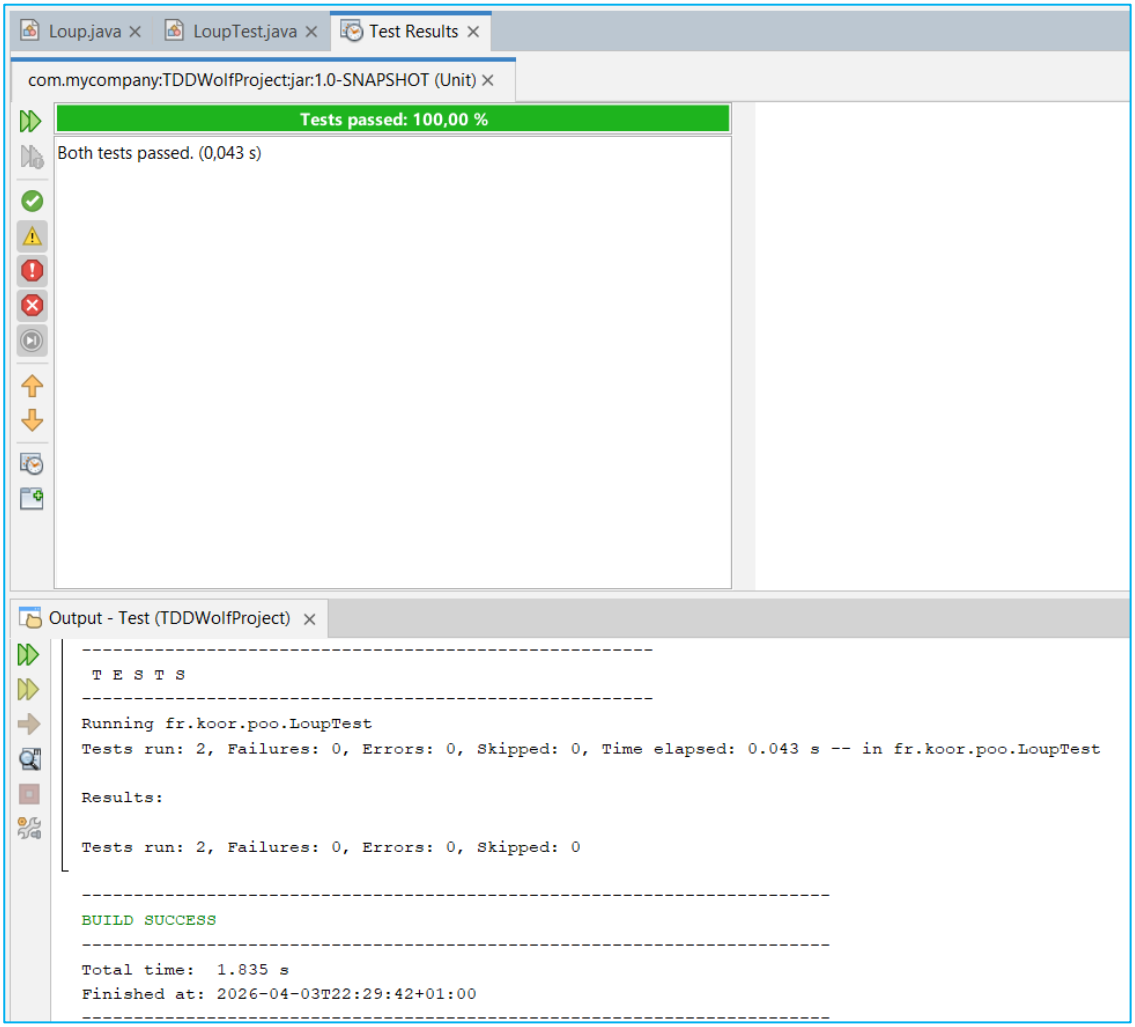
1. **A new test will be added to LoupTest**:



Execution: The code will fail (red)

2. Go to the **Loup** class and add the **tourner()** function to give the desired result:





Phase 4: **Refactoring**:

Instead of using a lot of repetitive if-else statements, we will use **Switch Expression**:

Loup.java × LoupTest.java ×

Source History

```java
package fr.koor.poo;

/**
 *
 * @author ahmed
 */

public class Loup {
    private Orientation orientation = Orientation.NORD;

    public Orientation getOrientation() {
        return orientation;
    }
    public void tourner() {
    this.orientation = switch (this.orientation) {
        case NORD  -> Orientation.EST;
        case EST   -> Orientation.SUD;
        case SUD   -> Orientation.OUEST;
        case OUEST -> Orientation.NORD; };
    }
}
```

The code is now very clear and easy to read, and will ensure coverage of all cases.

Loup.java × LoupTest.java × Test Results ×

com.mycompany:TDDWolfProject:jar:1.0-SNAPSHOT (Unit) ×

Tests passed: 100,00 %

Both tests passed. (0,037 s)

Output - Test (TDDWolfProject) ×

```
-------------------------------------------------------
 T E S T S
-------------------------------------------------------
Running fr.koor.poo.LoupTest
Tests run: 2, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.037 s -- in fr.koor.poo.LoupTest

Results:

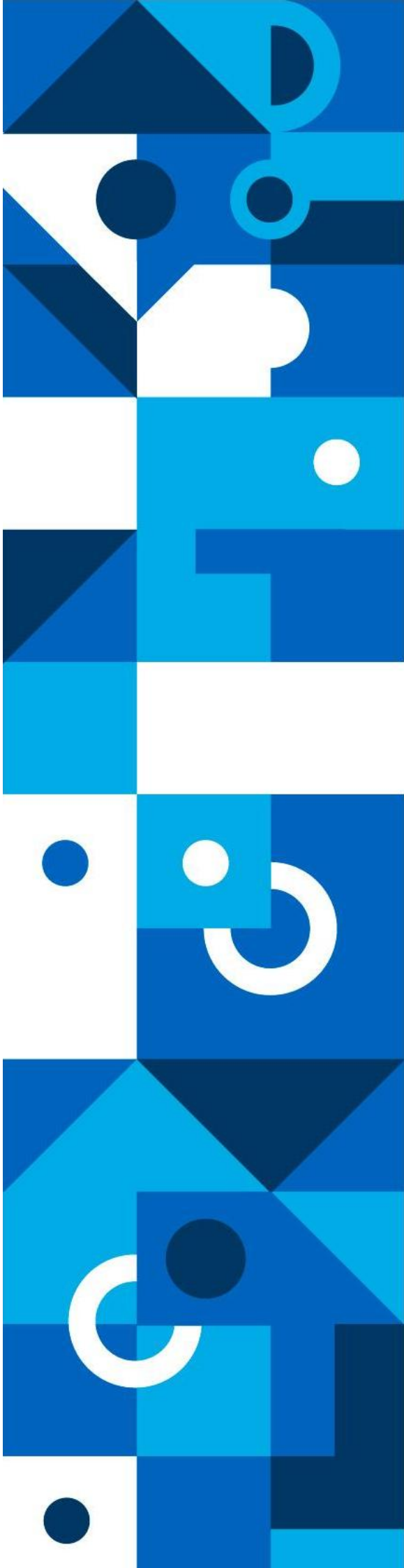
Tests run: 2, Failures: 0, Errors: 0, Skipped: 0

------------------------------------------------------------------------
BUILD SUCCESS
------------------------------------------------------------------------
Total time:  1.943 s
Finished at: 2026-04-03T22:42:19+01:00
------------------------------------------------------------------------
```

Conclusion

Test-Driven Development (TDD) is a sophisticated software engineering practice that prioritizes reliability and design from the very beginning. By adopting the "test-first" approach, developers can ensure that every feature is precisely implemented according to requirements while maintaining a clean and maintainable codebase. Ultimately, TDD empowers programmers to build robust Java applications with the confidence that new changes will not compromise existing functionality.

TP3: Automated testing with JUnit and Spring



Introduction to Testing in Spring Architecture

In a standard professional architecture, the **Service Layer** (containing business logic) and the **Data Access Layer** (DAO) (containing database queries like JDBC or Hibernate) are strictly separated.

- **Transactions** are typically managed by the Service Layer.
- **Spring Framework** allows automatic transaction management using annotations or programmatic approaches, with or without **Spring Boot**.

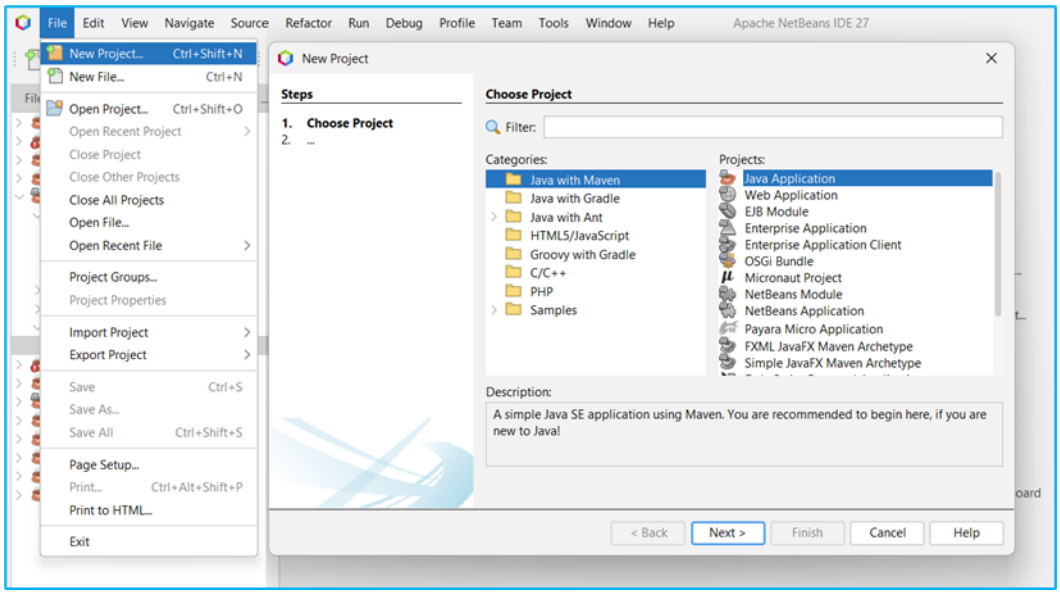
Practical Application:

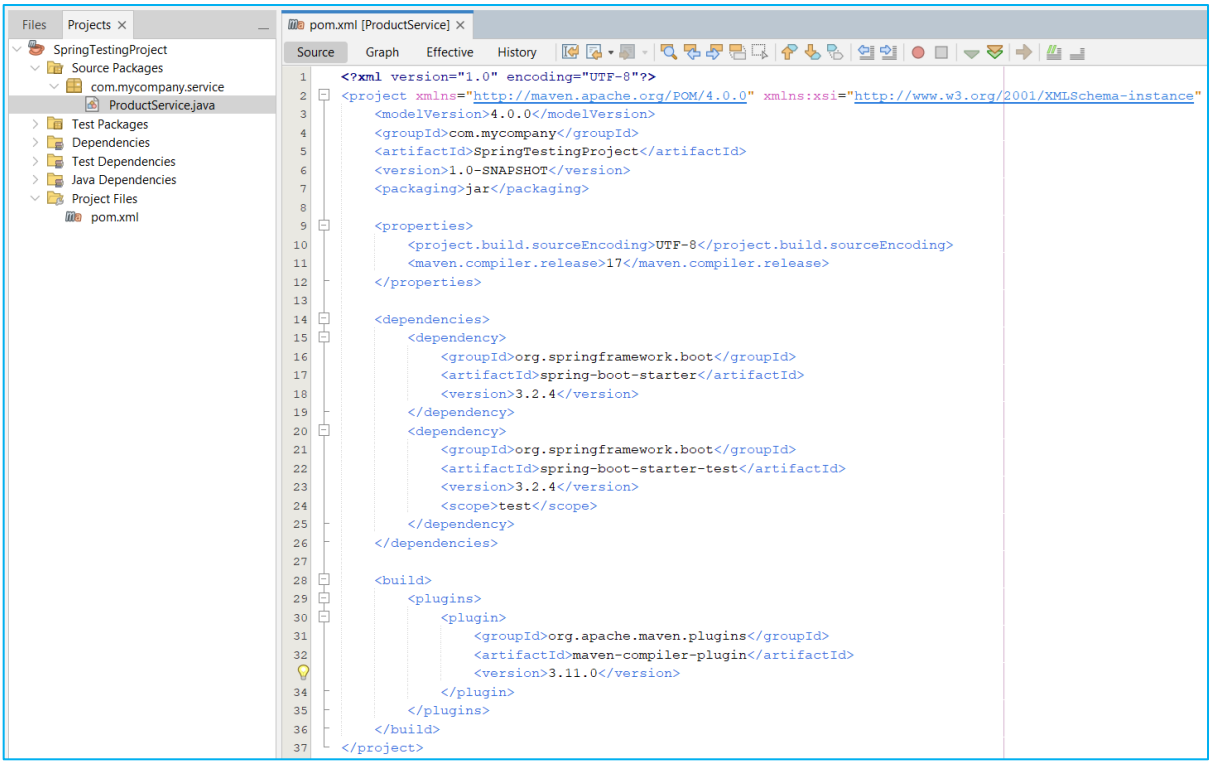
Testing with Spring Boot in NetBeans

Integrating JUnit 5 with the Spring Framework allows for robust integration testing. The following steps outline the practical implementation in a NetBeans Maven environment:

1. **Dependency Configuration**:

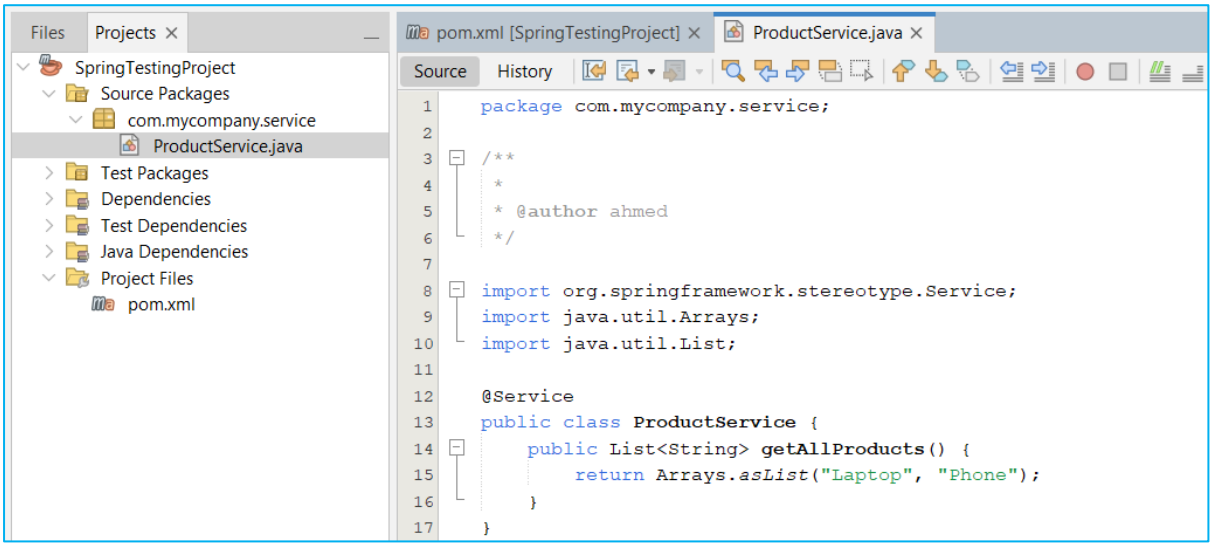
The **spring-boot-starter-test** and **spring-boot-starter dependencies** are added to the **pom.xml** file. This bundle provides the necessary libraries for both the **Spring core** and the testing suite (**JUnit 5**, **Mockito**, etc.).





2. Component Implementation:

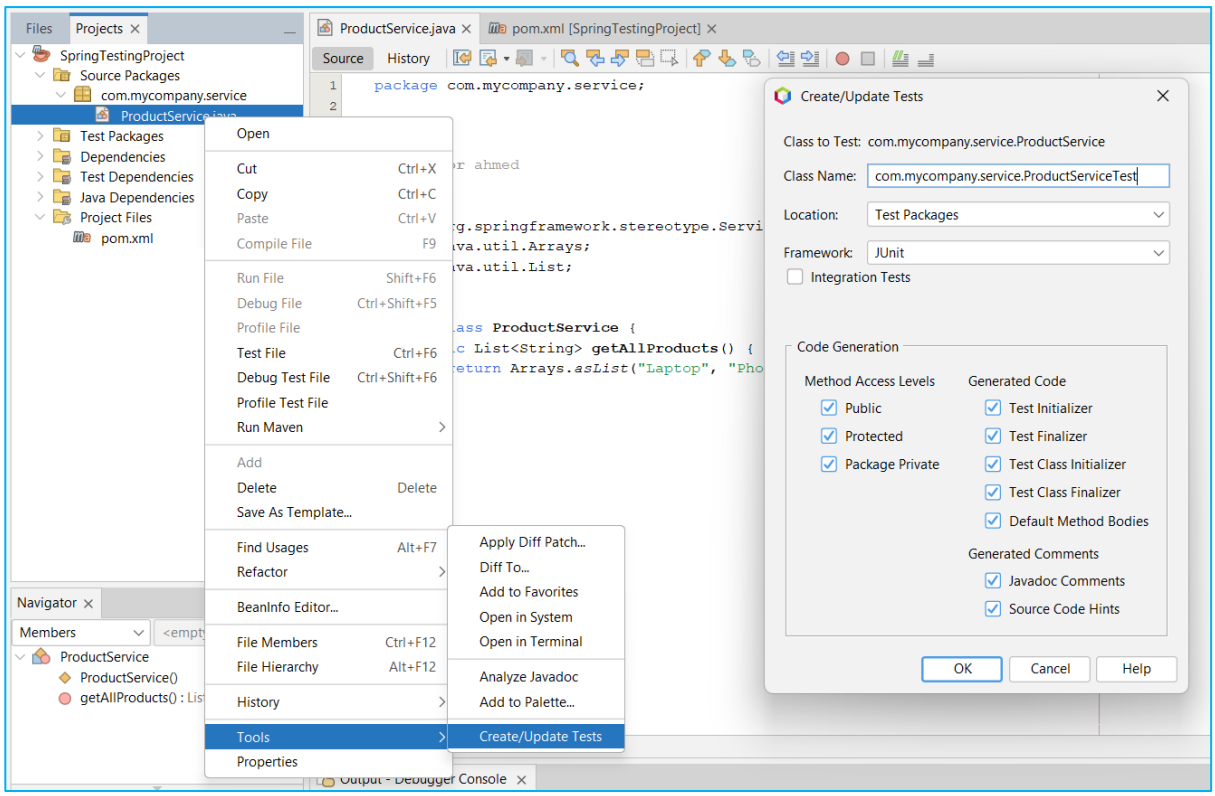
A service class (e.g., **ProductService**) is created and annotated with **@Service**. This informs **Spring** that the class is a **"Bean"** or a managed component that can be automatically instantiated and managed within the application context.

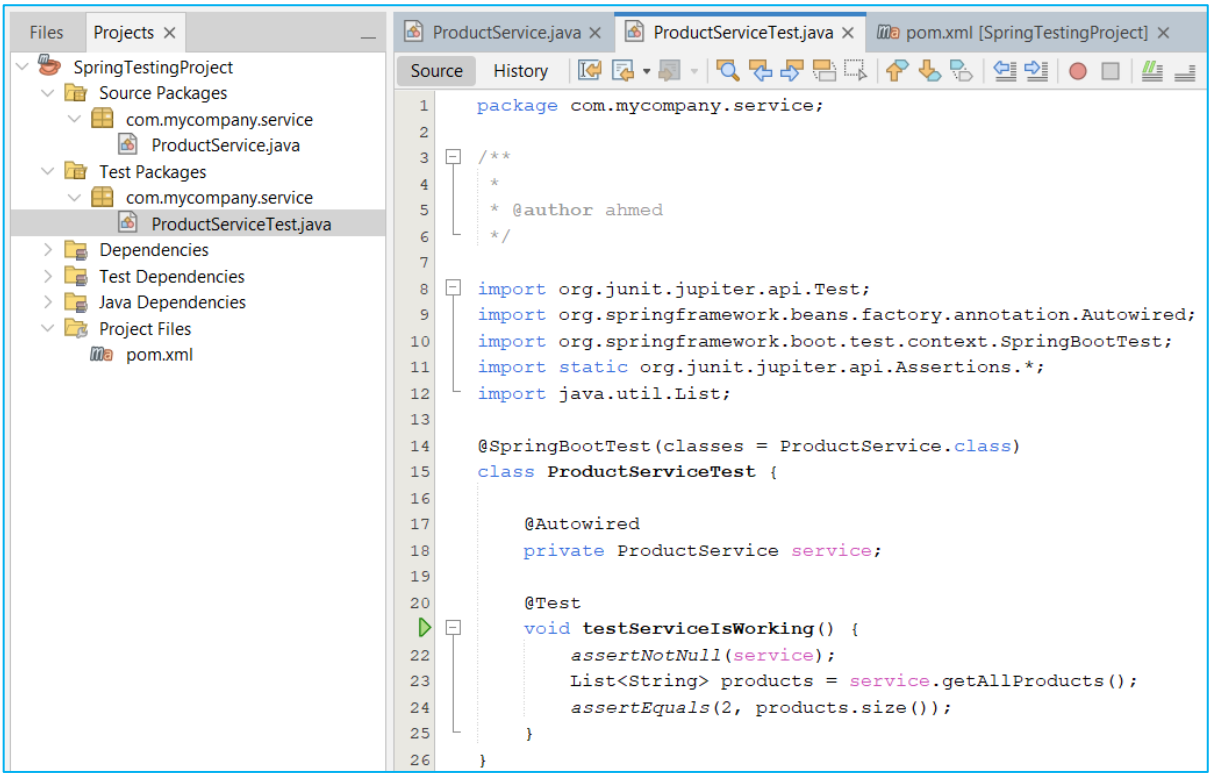


3. Dependency Injection (@Autowired):

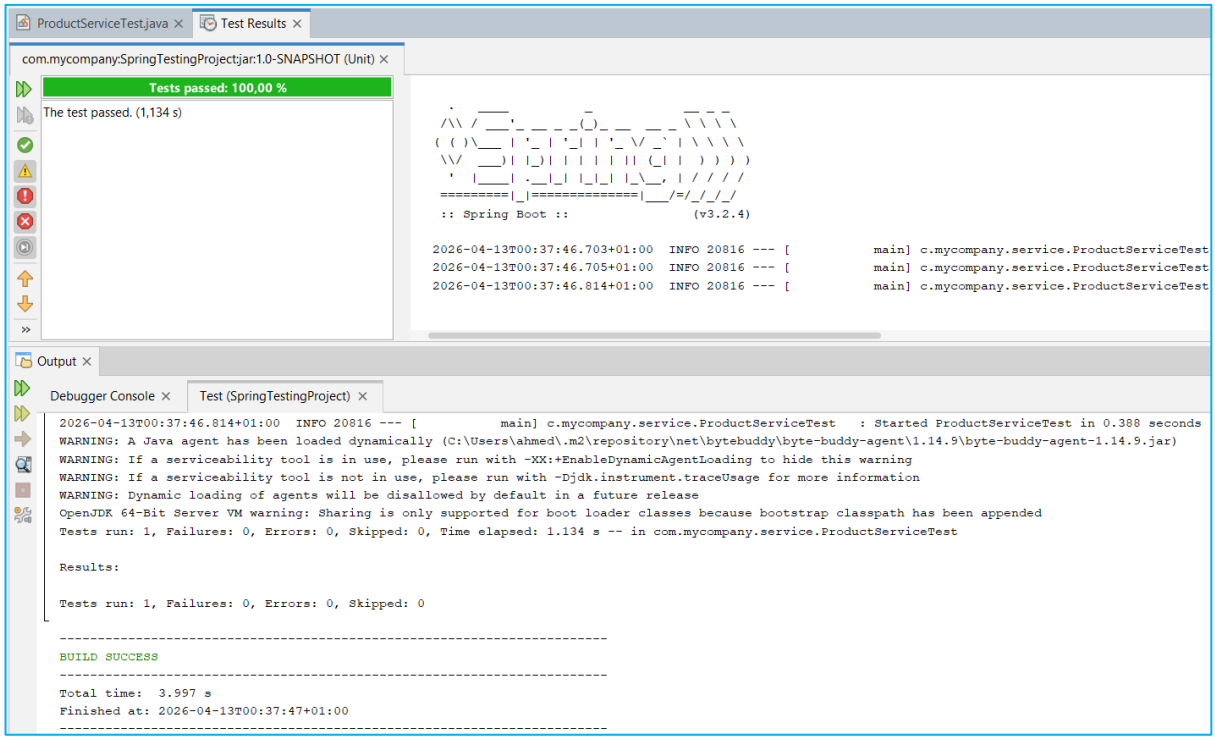
Instead of manually instantiating services, we use the **@Autowired** annotation to inject the **ProductService** bean directly into the test class. This ensures that

the service is tested with all its internal dependencies correctly managed by Spring.





Execution: Tests are executed using **Alt + F6**. NetBeans displays the Spring Boot startup logs in the Output window, followed by the test results, ensuring that the entire component chain is functioning correctly.



Transactional Testing with @Transactional:

In enterprise applications, tests often interact with a database. To maintain data integrity and isolation, the **@Transactional** annotation is applied to the test class. This ensures that every database operation performed during a test is automatically rolled back upon completion. This prevents tests from polluting the database with **"garbage" data** and ensures that each test starts with a consistent and predictable state.

1- Dependency Management:

The **spring-boot-starter-data-jpa** dependency was added to **pom.xml** to provide the necessary transaction management infrastructure.

To support database operations during testing, the **H2 Database** (in-memory) dependency was included. This allows Spring to simulate a database environment without requiring an external SQL server installation.

```xml
<dependencies>
    <dependency>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter</artifactId>
        <version>3.2.4</version>
    </dependency>
    <dependency>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-data-jpa</artifactId>
        <version>3.2.4</version>
    </dependency>
    <dependency>
        <groupId>com.h2database</groupId>
        <artifactId>h2</artifactId>
        <version>2.2.224</version>
        <scope>runtime</scope>
    </dependency>
    <dependency>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-test</artifactId>
        <version>3.2.4</version>
        <scope>test</scope>
    </dependency>
</dependencies>
```

2- Application Context Configuration:

A main application class, **SpringTestingProjectApplication**, was created with the **@SpringBootApplication** annotation. This class serves as the "root" for Spring's component scanning, enabling the test framework to locate and initialize the **@Service** beans (like **ProductService**).

SpringTestingProjectApplication.java × pom.xml [SpringTestingProject] ×

Source History

```java
package com.mycompany;

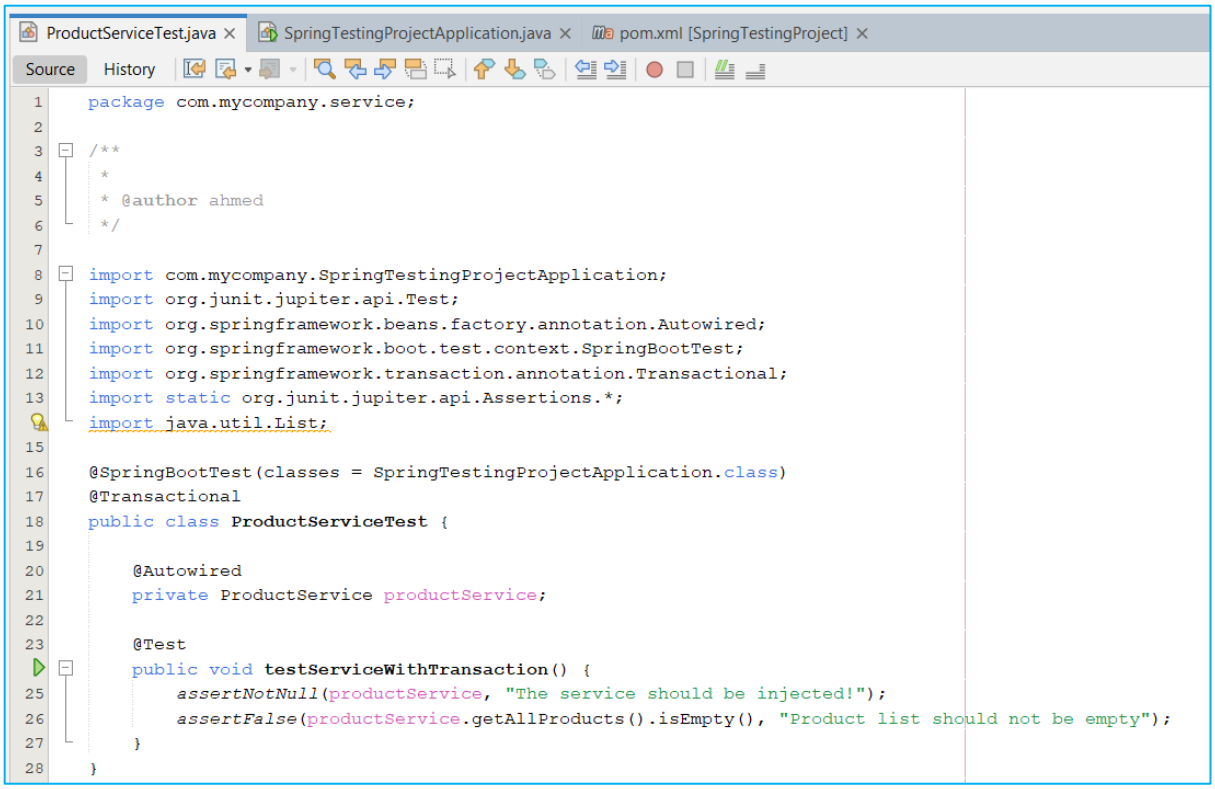
import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;

@SpringBootApplication
public class SpringTestingProjectApplication {
    public static void main(String[] args) {
        SpringApplication.run(SpringTestingProjectApplication.class, args);
    }
}
```

3- Applying the @Transactional Annotation:

The **@Transactional** annotation was applied at the class level of the **ProductServiceTest**. This informs the **Spring TestContext** Framework that

every test method within this class should be executed within a managed database transaction.



Test Class Integration: The test class was annotated with **@SpringBootTest(classes = SpringTestingProjectApplication.class)**. By explicitly defining the classes parameter, we ensured that the test environment correctly loads the application's configuration even when files are located in different packages.

Automatic Rollback: By default, Spring's testing framework automatically triggers a **Rollback** at the end of each test method. This ensures that any data created or modified during the test is never committed to the database, keeping the environment consistent for subsequent tests.

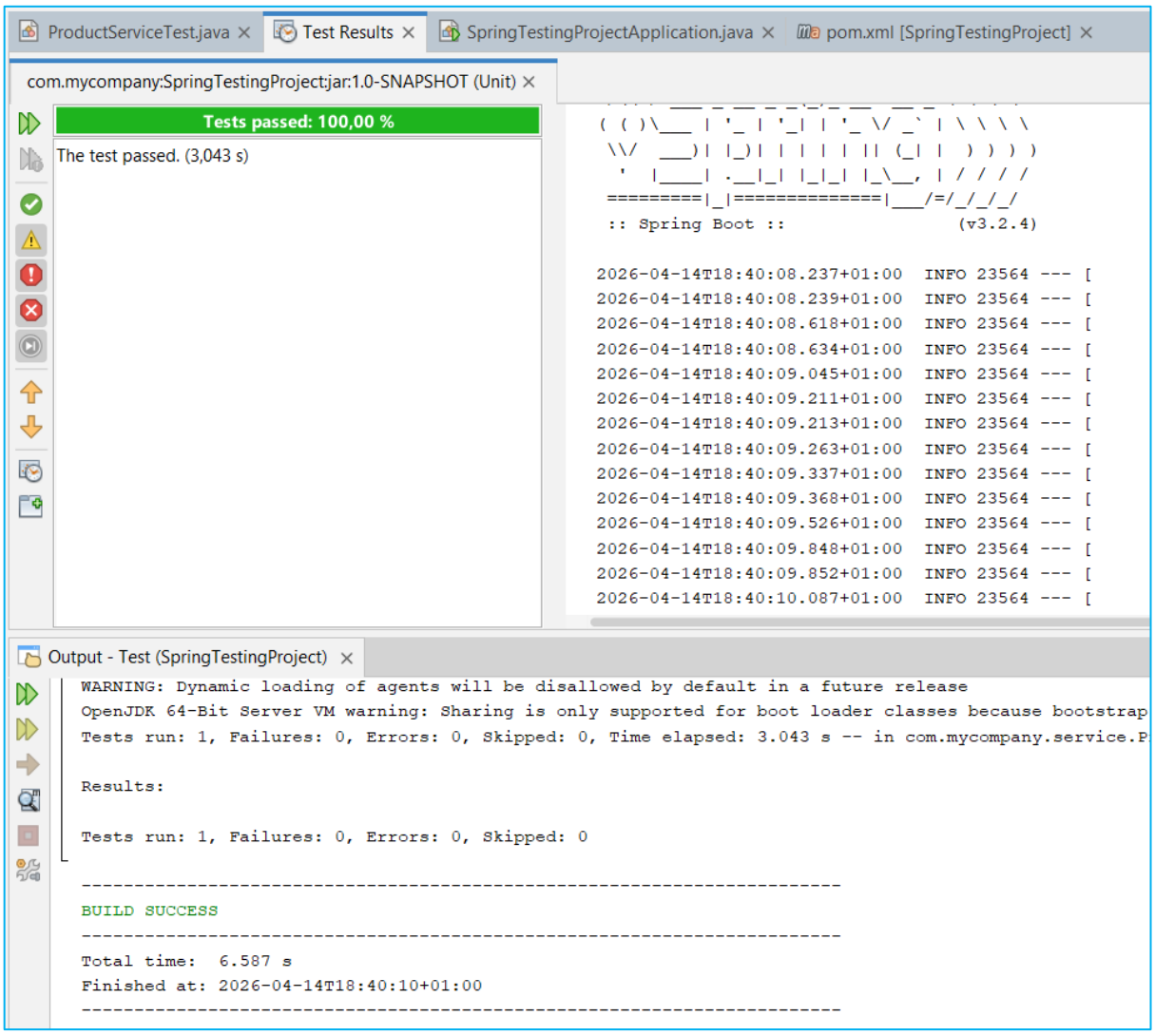
4- **Verification through Logs**:

Upon execution (using **Alt + F6**), the Spring Boot console logs confirm the transaction management by displaying messages such as:

1- Began transaction for test context...

2- Rolled back transaction for test context...

Benefits Realized: This approach eliminates the need for manual database cleanup and prevents **"dirty data"** from affecting the reliability of the test suite, resulting in faster and more predictable automated tests.

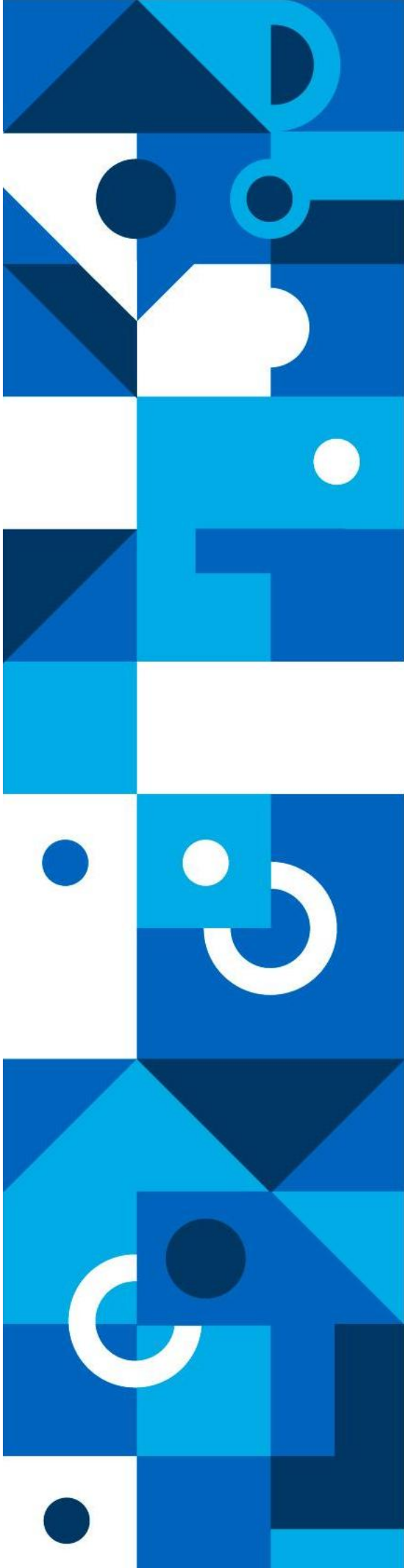


Conclusion

Testing in a Spring environment transition from simple unit tests to comprehensive **Integration Tests**. By leveraging Spring's robust testing support—specifically **@SpringBootTest** for context loading and **@Transactional** for automatic rollback—developers can ensure that business logic and database

interactions work seamlessly together. This approach guarantees a clean, automated, and isolated test suite, ensuring that the application remains reliable and production-ready without the overhead of manual database management.

TP4: Integration tests and BDD methodology



Introduction

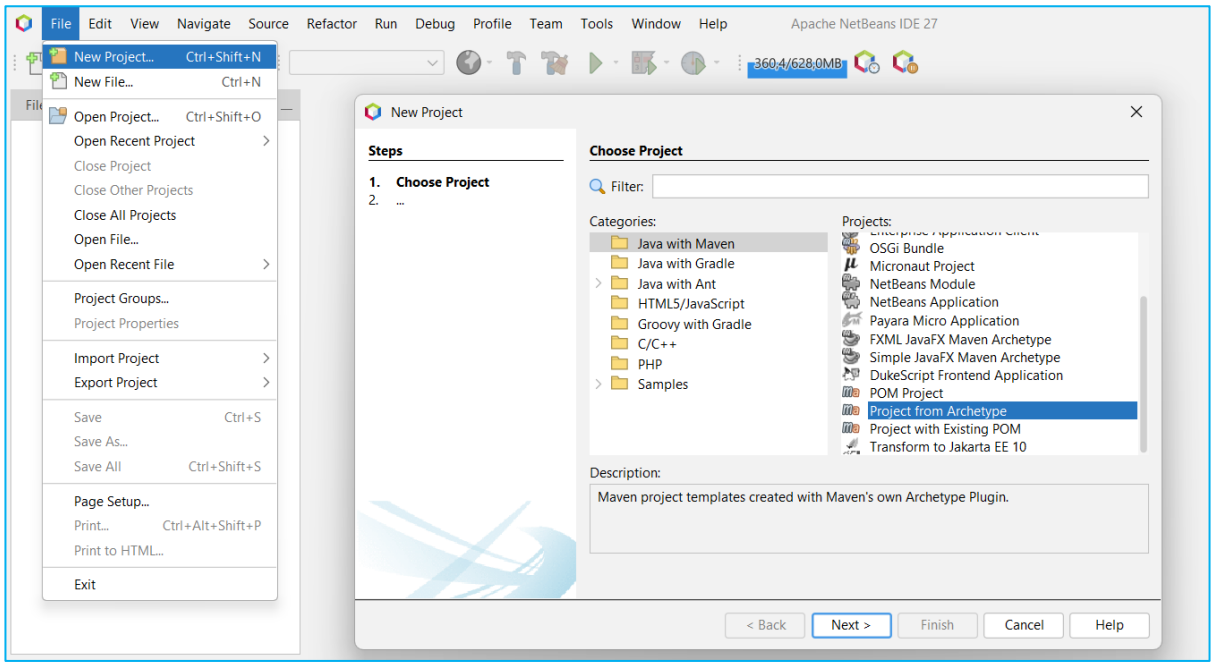
In modern software engineering, ensuring that individual modules work harmoniously is critical. We will explore **Integration Testing** through the lens of **Behavior-Driven Development (BDD)**. By leveraging the **Cucumber-JVM** framework, developers can bridge the communication gap between technical teams and business stakeholders, translating plain-language requirements into executable, automated test suites.

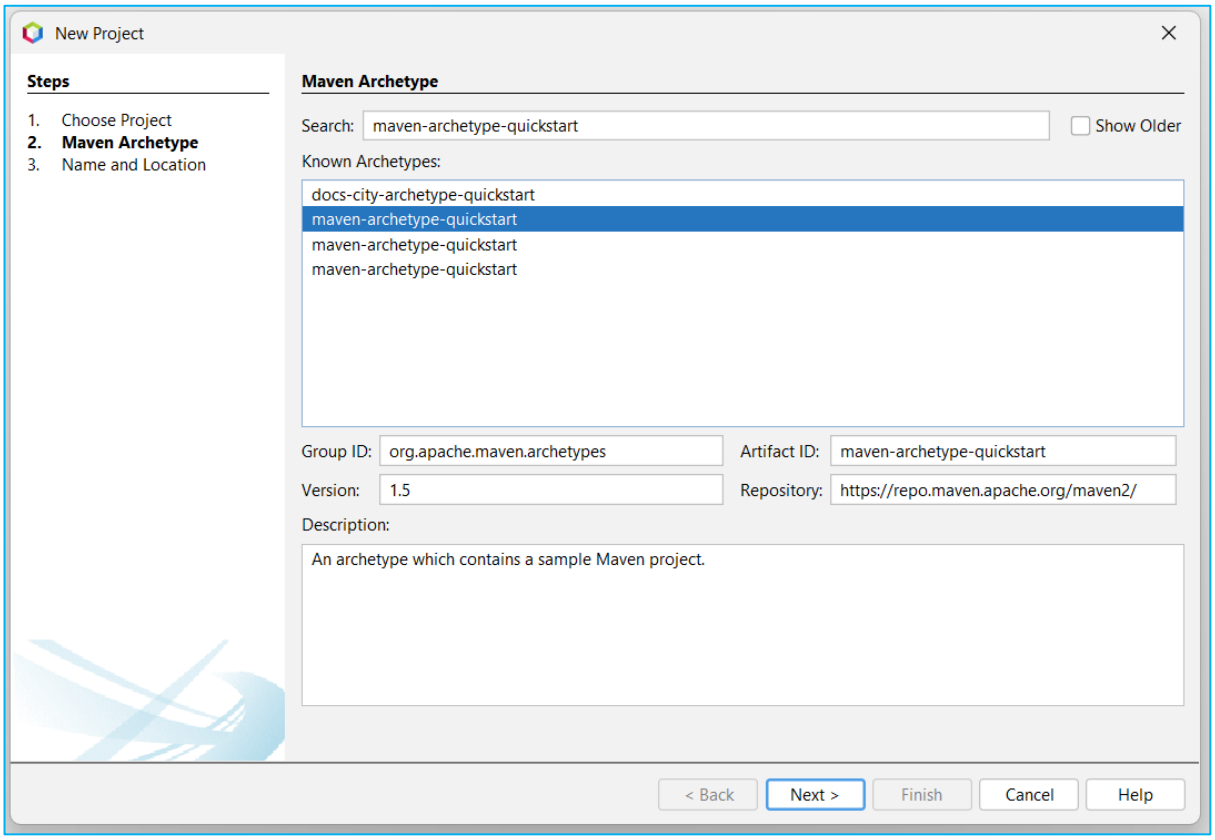
Practical Steps

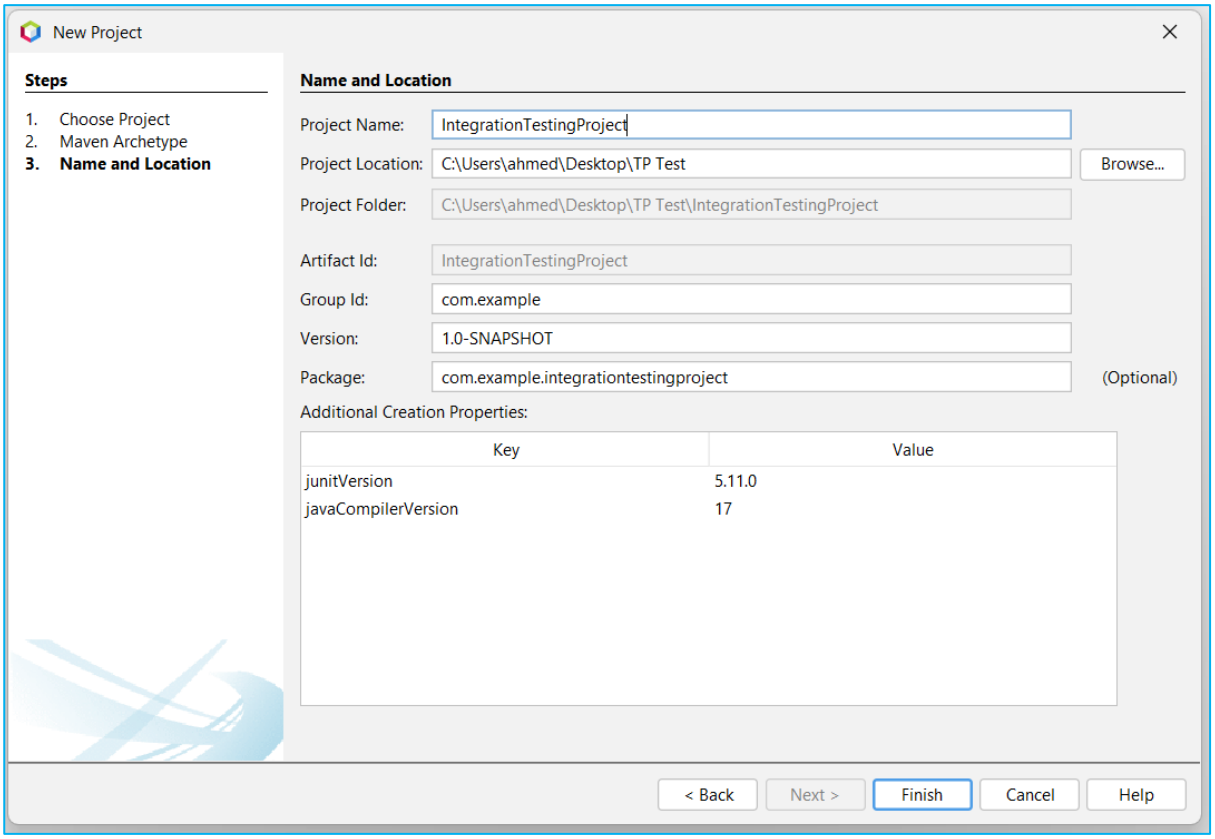
> **Note**: these **Maven-based** steps are fully compatible with **NetBeans** and other major IDEs.

1- **Project Architecture:**

Utilizing the **maven-archetype-quickstart** to create a standardized directory structure that adheres to the "**Convention over Configuration**" principle.







2- **Dependency Management:**

Integrating core libraries within the **pom.xml** file, specifically **cucumber-java** for the **BDD engine** and **cucumber-junit** for test execution and reporting.

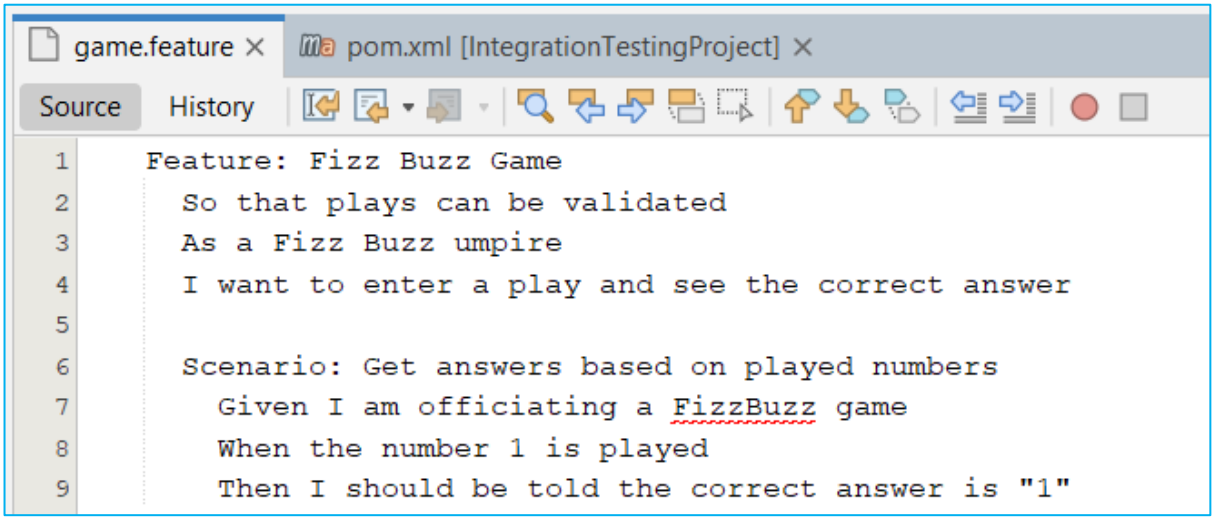


3- **Resource Organization:**

Creating a dedicated **src/test/resources** directory to house **Gherkin feature** files, ensuring a clean separation between testing logic and test specifications.

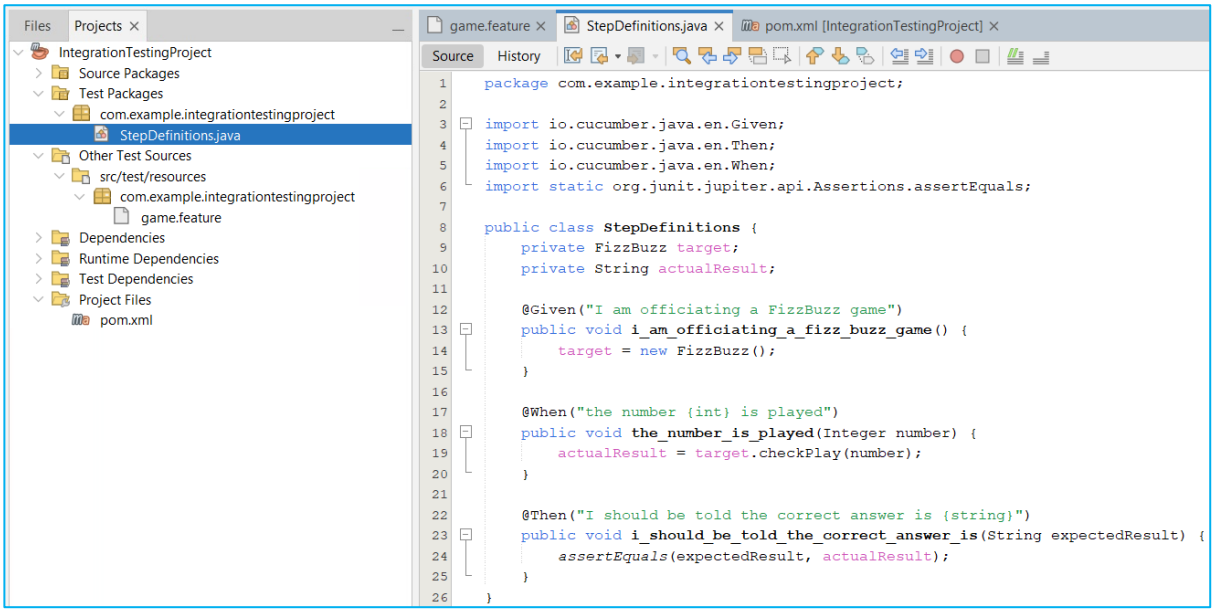
- **Script file (game.feature):**

This file describes the game's behavior in simple language that everyone can understand.

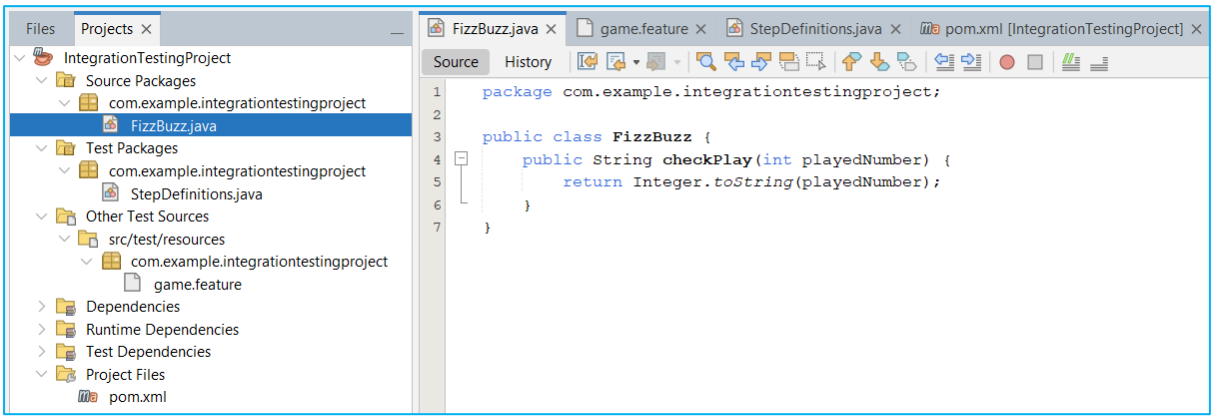


- **The programming link file (StepDefinitions.java):**

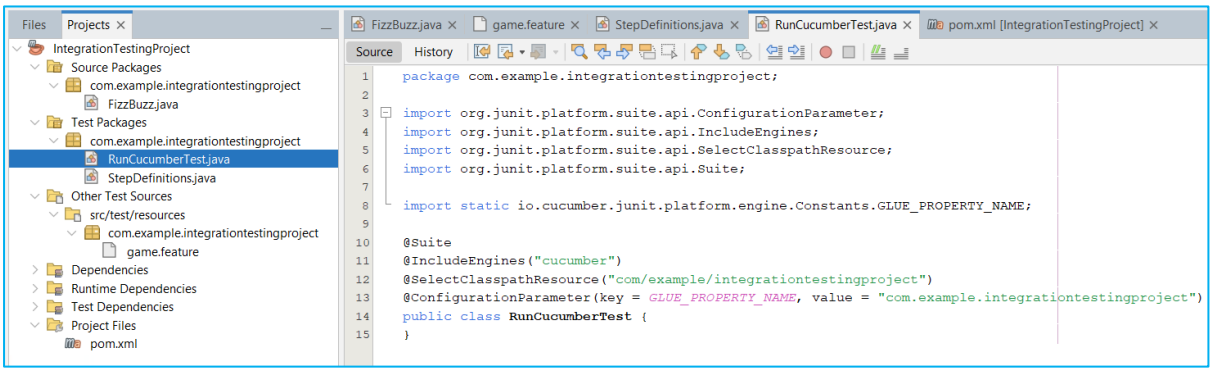
This code links the statements written in the Feature file to the programming logic in Java.



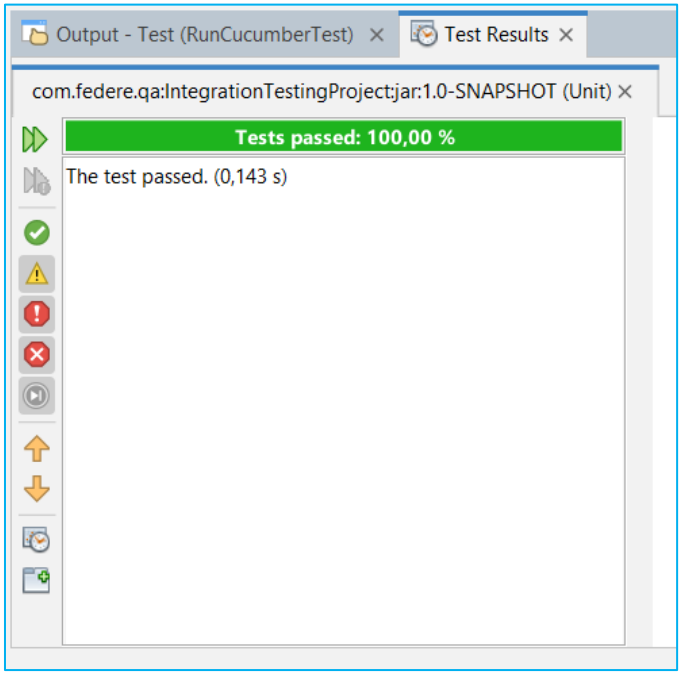
- **The Programming logic file (FizzBuzz.java):**



- **The Test Runner file (RunCucumberTest.java):**



- **Execution:** When you run the test, a window called **Test Results** will appear, and you will see a green bar if everything is correct.



4- **Developing the logic of the game (The Challenge):**

FizzBuzz.java

Source | History

```java
package com.example.integrationtestingproject;

public class FizzBuzz {
    public String checkPlay(int playedNumber) {
        if (playedNumber % 3 == 0 && playedNumber % 5 == 0) return "FizzBuzz";
        if (playedNumber % 3 == 0) return "Fizz";
        if (playedNumber % 5 == 0) return "Buzz";
        return Integer.toString(playedNumber);
    }
}
```

5- **Adding new scenarios (Scaling):**

game.feature

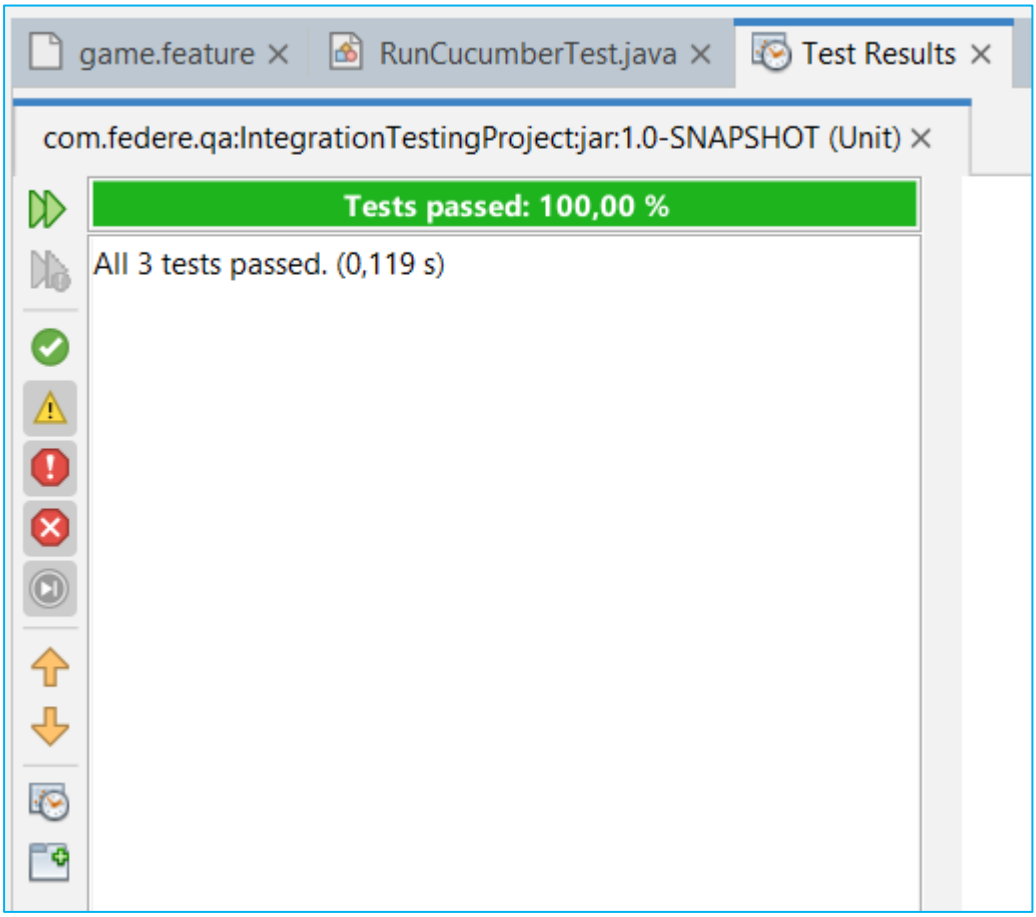
Source | History

```
Feature: Fizz Buzz Game
  So that plays can be validated
  As a Fizz Buzz umpire
  I want to enter a play and see the correct answer

  Scenario: Get answers based on played numbers
    Given I am officiating a FizzBuzz game
    When the number 1 is played
    Then I should be told the correct answer is "1"
  Scenario: Testing Fizz rule
    Given I am officiating a FizzBuzz game
    When the number 3 is played
    Then I should be told the correct answer is "Fizz"

  Scenario: Testing Buzz rule
    Given I am officiating a FizzBuzz game
    When the number 5 is played
    Then I should be told the correct answer is "Buzz"
```

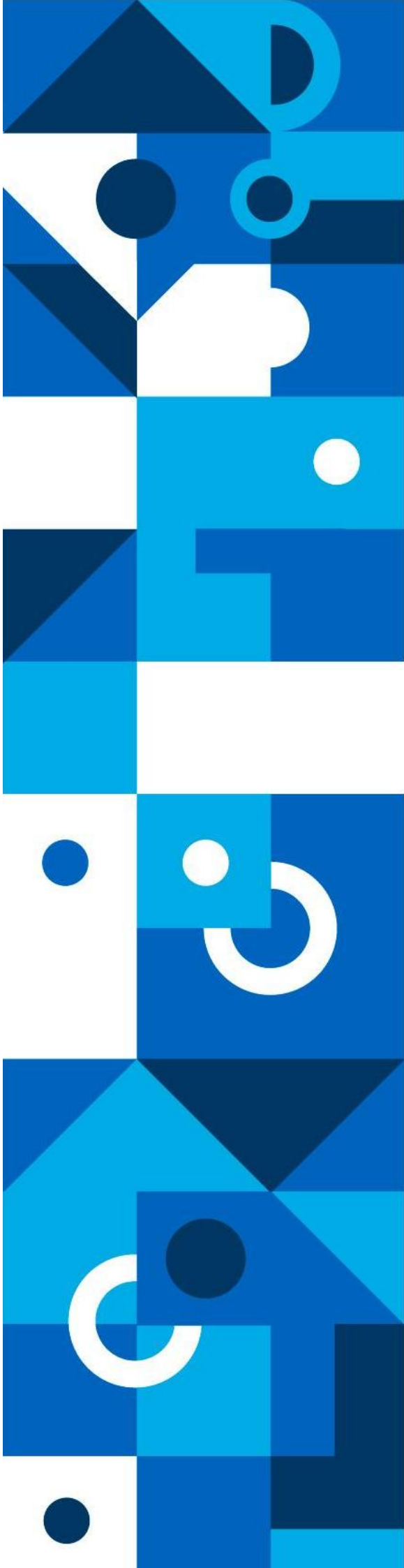
6- **Execution:** Run the **RunCucumberTest.java** file again



Conclusion

The shift toward **BDD** and **automated integration testing** represents a significant leap in software quality assurance. By utilizing **Gherkin-based scenarios**, we bridge the gap between technical implementation and business requirements. While tools like **ChatGPT** can drastically accelerate the generation of initial test scripts, the human element remains indispensable for architectural decisions, security enforcement, and the maintenance of a resilient **CI/CD pipeline**. This comprehensive approach ensures that software is not just functional, but also secure and maintainable in the long term.

TP5: ChatGPT to facilitate software test automation



Introduction

As software delivery cycles accelerate, AI-driven tools like **ChatGPT** are emerging as critical force multipliers for **Quality Assurance teams**. This section examines how conversational AI facilitates "**shift-left**" testing by automating the creation of comprehensive test scenarios, reducing the overhead of script maintenance, and architecting robust testing infrastructures. We explore the tangible efficiency gains offered by these tools while addressing the critical limitations and "hallucinations" that require human oversight, offering a realistic roadmap for AI-augmented test automation abilities.

Objectives and Initial Use Cases

The primary objective is to leverage **ChatGPT** to simplify and enhance the workflow of web and mobile application testers, with a particular focus on test automation. The initial use cases involve generating basic test cases for common scenarios, such as search functionality (e.g., **Google**) and user authentication processes (e.g., **login forms**). By harnessing generative AI, the goal is to accelerate the test development lifecycle, especially during the initial "**scaffolding**" phase. This includes supporting widely used automated testing frameworks and technologies, enabling testers to quickly build, structure, and execute their test suites more efficiently.

Building Complex Automation Pipelines

The use of **ChatGPT** enables the design and implementation of more advanced test automation pipelines. This includes the integration of **CI/CD** workflows and the creation of **Bash scripts** to test applications composed of multiple microservices.

ChatGPT proves effective in generating well-structured and thoroughly documented code, accompanied by clear and actionable instructions. This approach makes complex automation processes more accessible, even for users

with limited experience in certain technologies, while also improving development speed and consistency.

Advantages of ChatGPT for Test Automation:

- **Adding Resilience**: Simplifies the creation of retry mechanisms, often overlooked by testers.
- **Ensuring Security**: Helps in implementing security best practices, such as using GitHub Secrets and environment variables.
- **Overcoming Learning Curves**: Acts as an excellent tool for testers to quickly get started with unfamiliar technologies (e.g., GitHub pipelines for Selenium experts).
- **Accelerating Code Debugging**: Can assist in identifying reasons for code failures, though not infallible.

Disadvantages of ChatGPT for Test Automation:

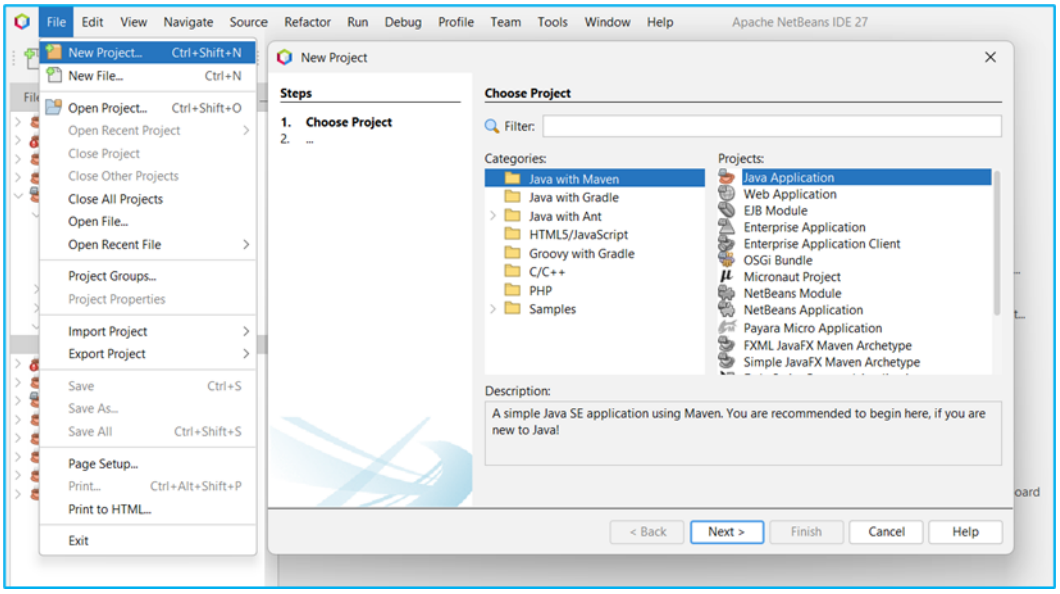
- **Statistical Model Limitations**: Being based on statistical models**, ChatGPT** may lack a deep understanding of underlying meaning, which can affect accuracy in complex, context-dependent scenarios.
- **Learning Gaps**: Its knowledge is based on its training data, meaning it may provide outdated information or incomplete code if its learning has not been updated.
- **Incomplete Code**: Often generates partially written code, requiring manual completion or modification, which can be challenging and error prone.

Practical Application:

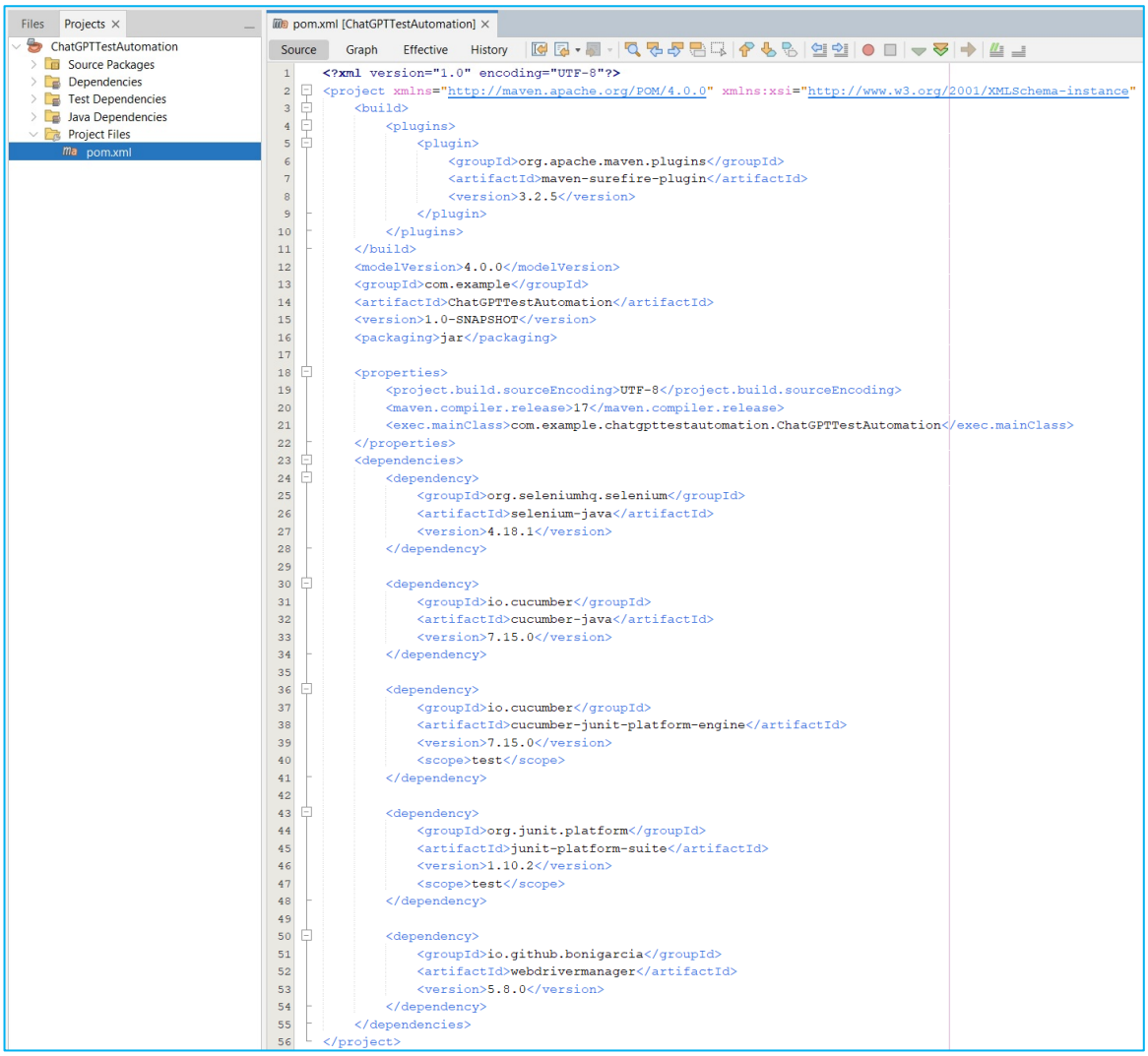
Selenium and Cucumber testing

To implement the practical aspects, we will set up a Maven project in NetBeans for **Selenium and Cucumber testing**. This section outlines the detailed steps:

1- Dependency Configuration:



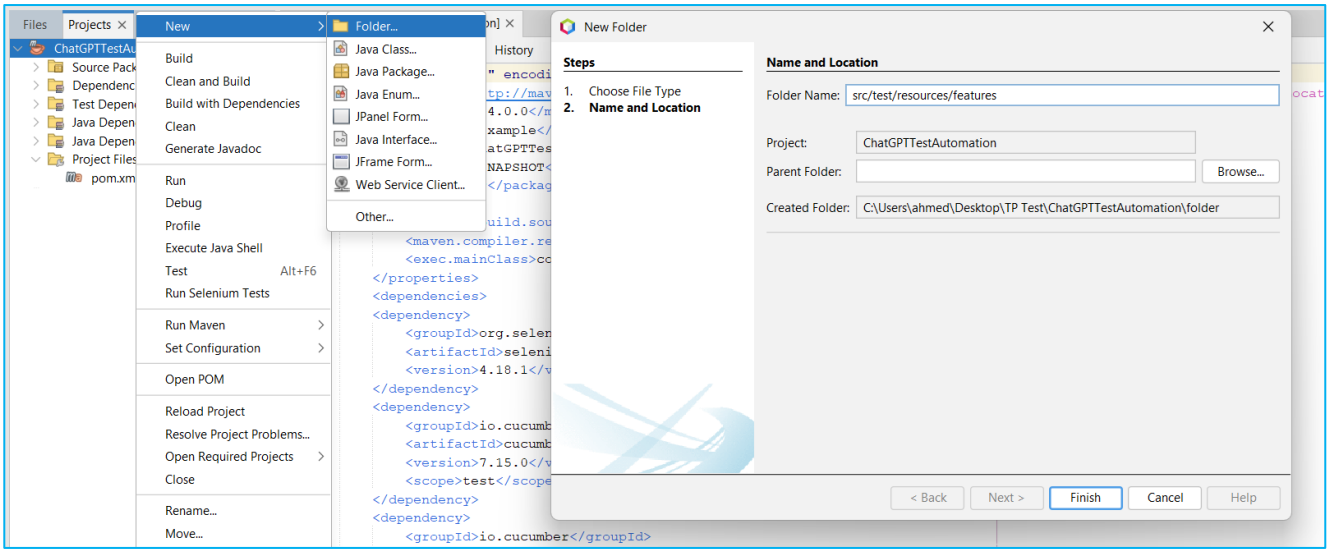
Once the project is created, you need to add the necessary dependencies to your **pom.xml** file. These typically include **Selenium**, **Cucumber**, and **JUnit**.



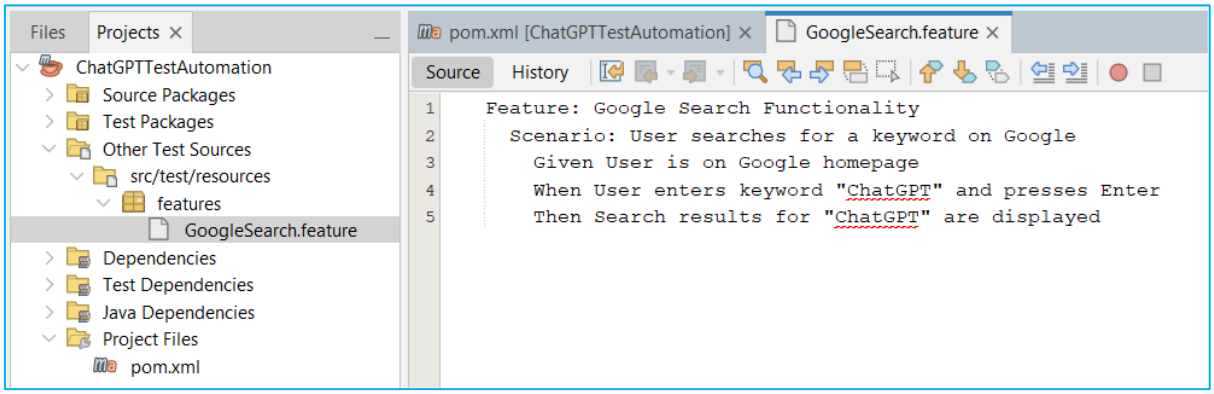
2- **Project Structure for Cucumber:**

For **Cucumber** tests, a typical project structure involves feature files, step definitions, and a test runner.

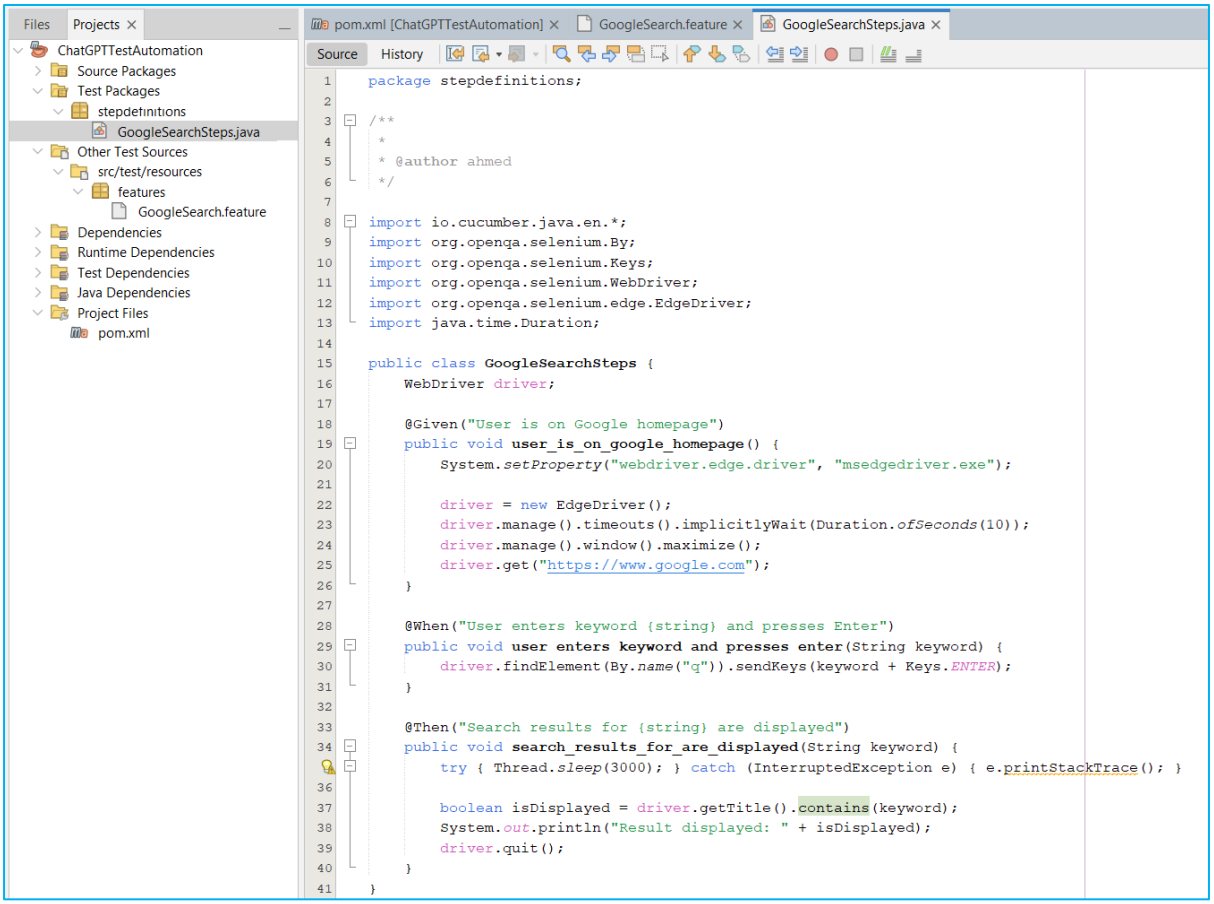
- **Create feature files:** Create a folder **src/test/resources/features**



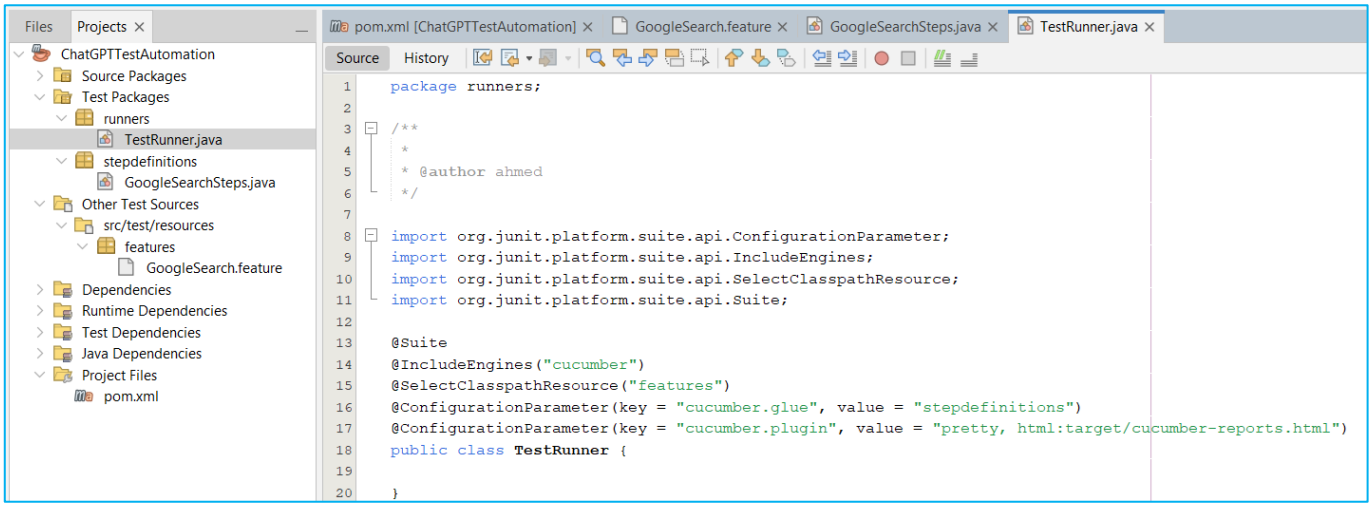
- **Create the File "GoogleSearch.feature":** Write a test plan in a **.feature file** using a simple language (**Gherkin**) to clearly describe what we are testing before programming.



- **Creating "Step Definitions":** Create **GoogleSearchSteps.java** class to link the feature file steps to **Selenium's programming commands**.



- **Create a TestRunner.java class:**

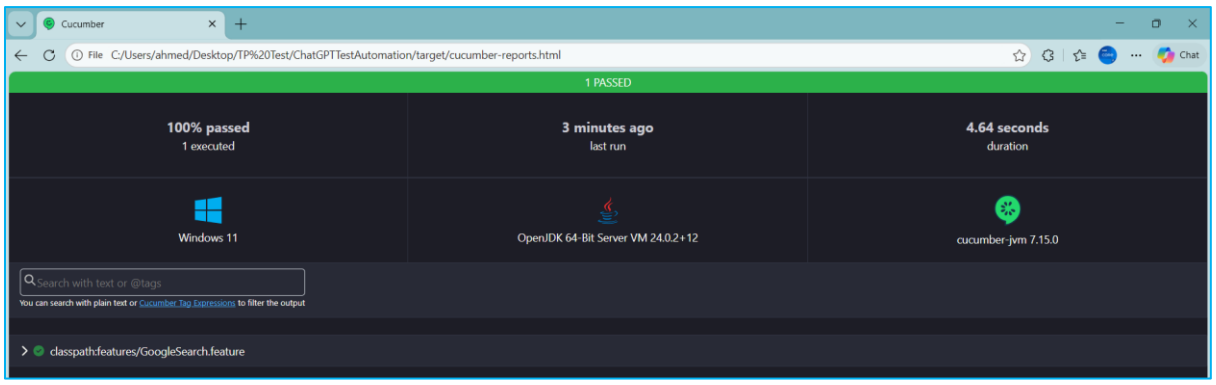


3- **Running Tests:**

Right-click on your project in NetBeans and select **Clean and Build** to download dependencies and compile your code.

The framework will automatically locate the **features** folder, read the **GoogleSearch.feature** file, map its steps to the corresponding methods in

GoogleSearchSteps.java, launch the browser, and execute the test automatically.



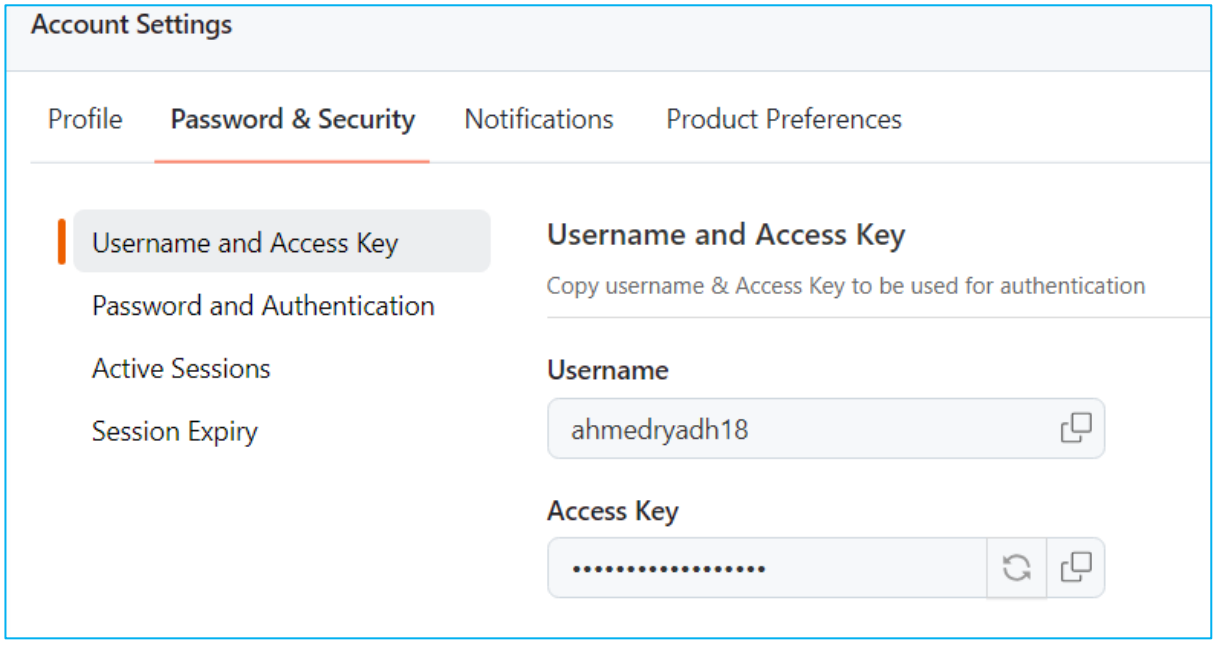
4- Integrating with LambdaTest:

Integration with **LambdaTest** means running your tests on the cloud instead of your local machine, allowing you to test across different browsers and operating systems without needing to install them.

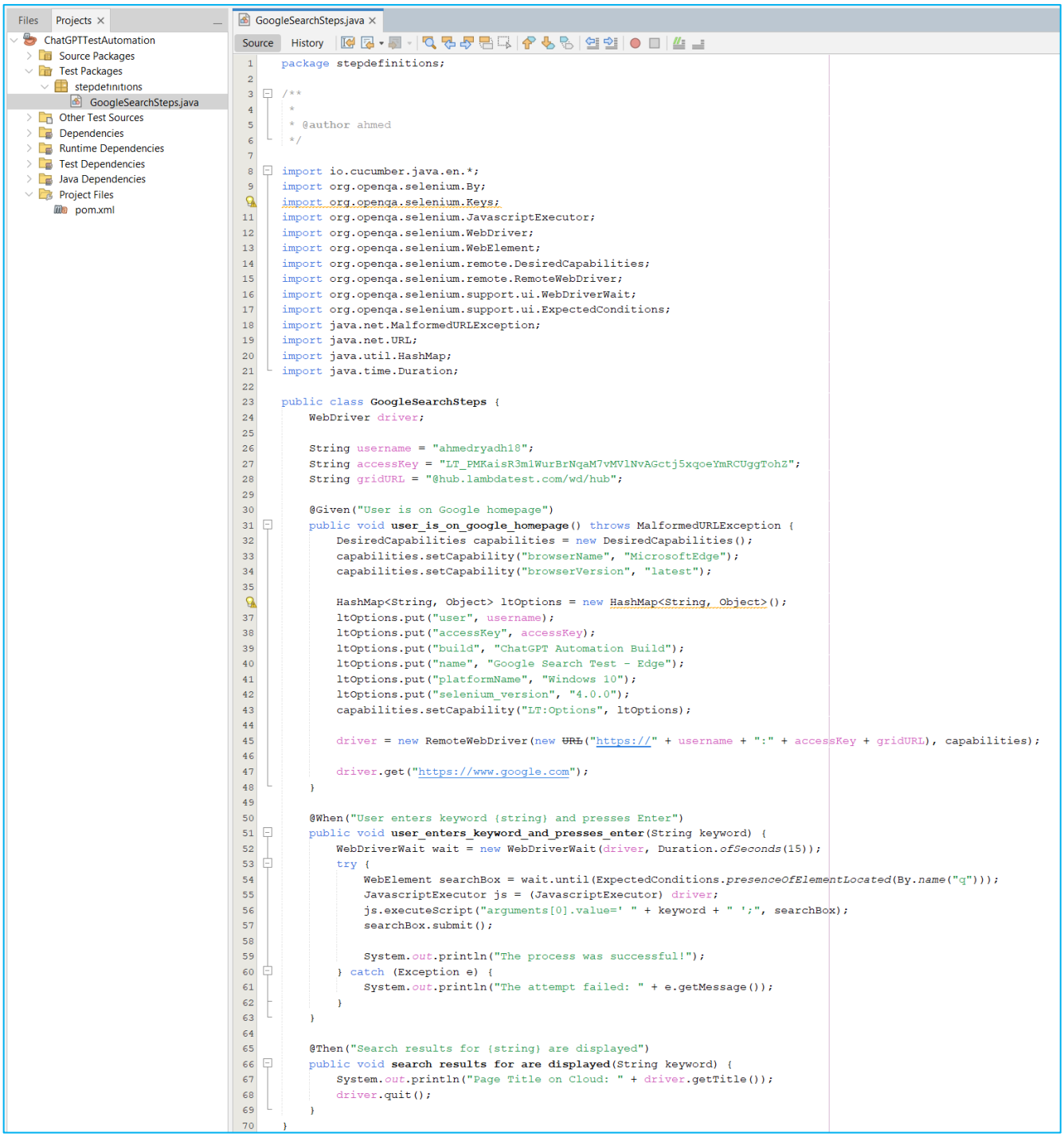
- **Obtaining credentials:**

You need to have an account on **LambdaTest.com.** From the Dashboard, retrieve the following details:

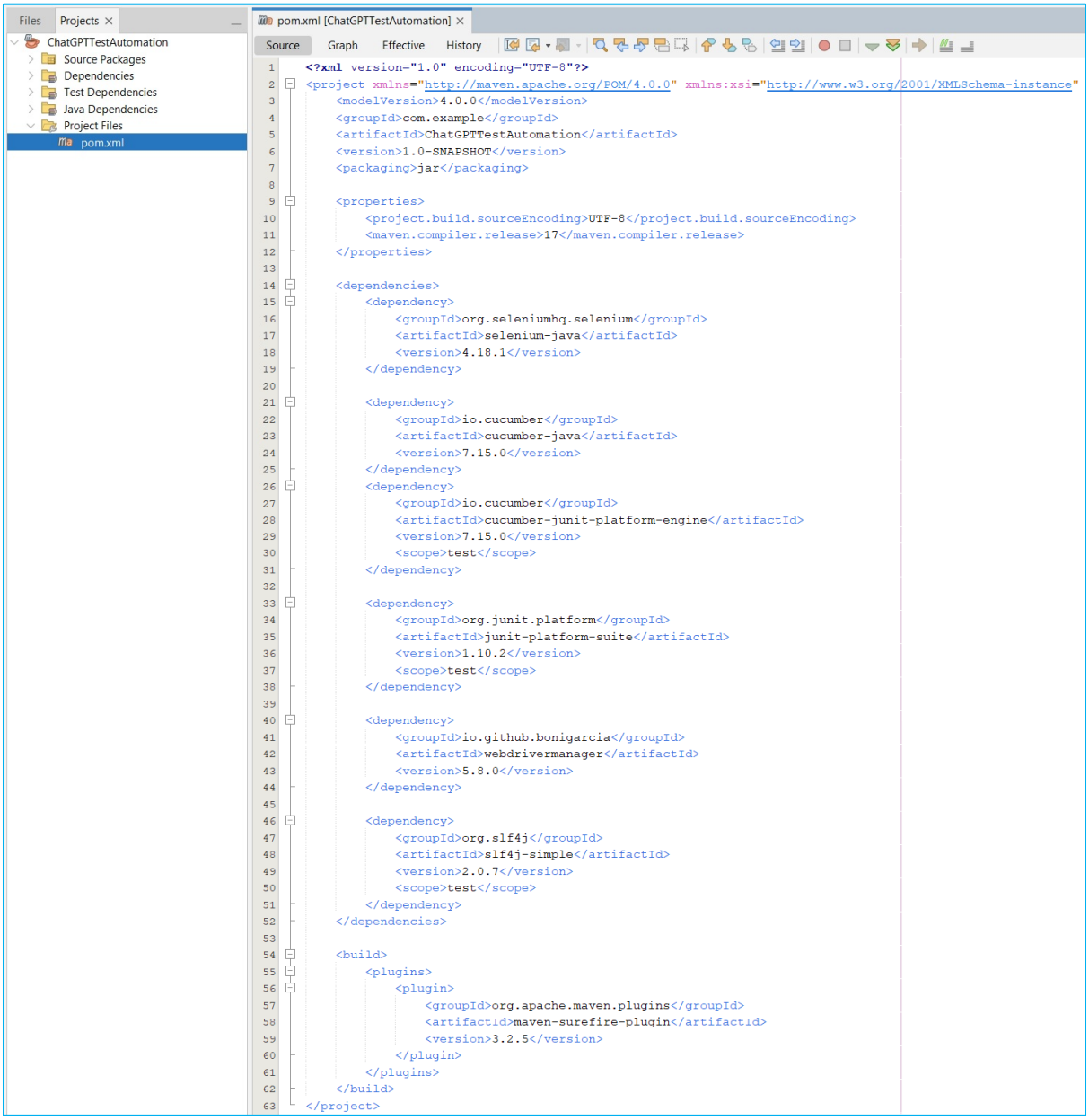
- **LT_USERNAME**: your username
- **LT_ACCESS_KEY**: your access key



- **Update StepDefinitions code:**



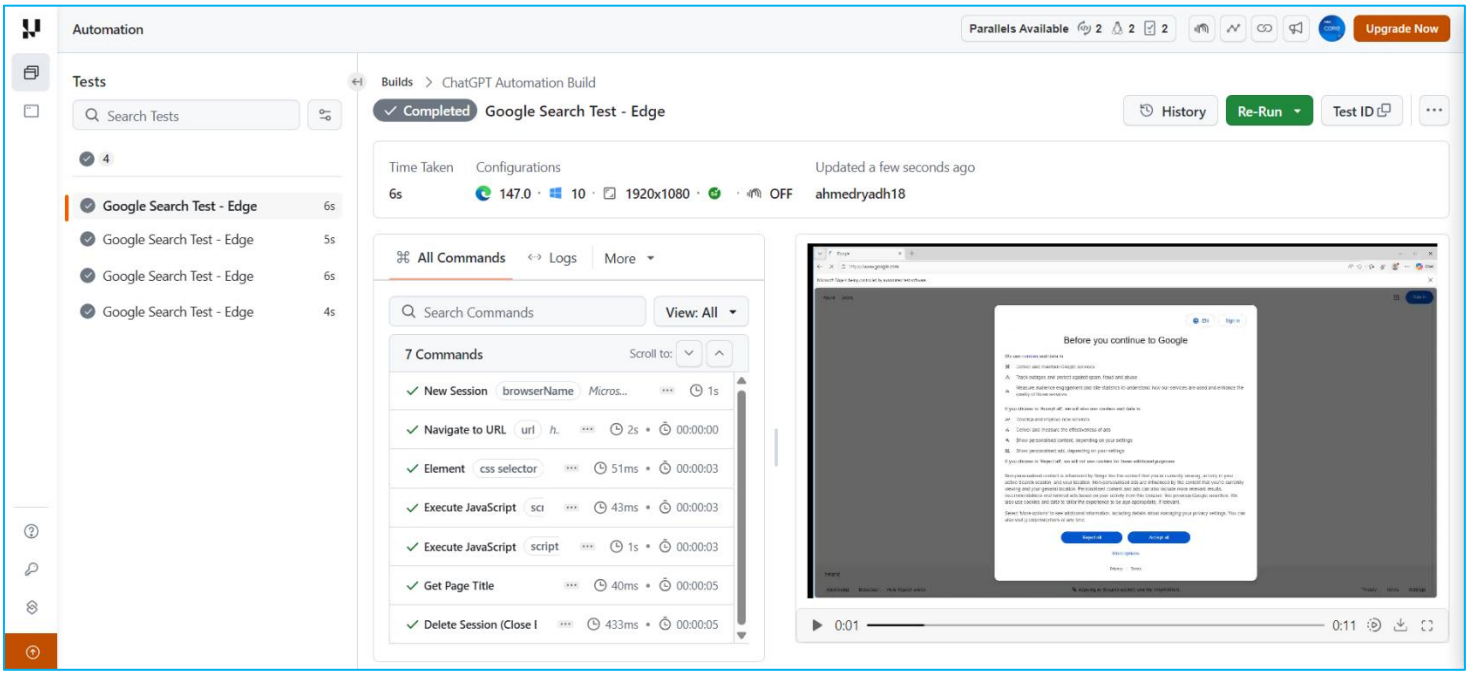
- **Changes in the pom.xml file:**



After modifying the file, make sure to press **Clean and Build** in NetBeans.

- **Execution**: When you execute the **TestRunner**, the Edge browser will not open on your local machine. Instead, the code sends the commands to LambdaTest.

You can log in to your **LambdaTest account** and watch a live stream of the test as it runs on their servers.



5- **Negative tests and exceptions:**

We will add a negative scenario that tests the search for something that does not exist, and we will develop the **exception handling** to be more stringent.

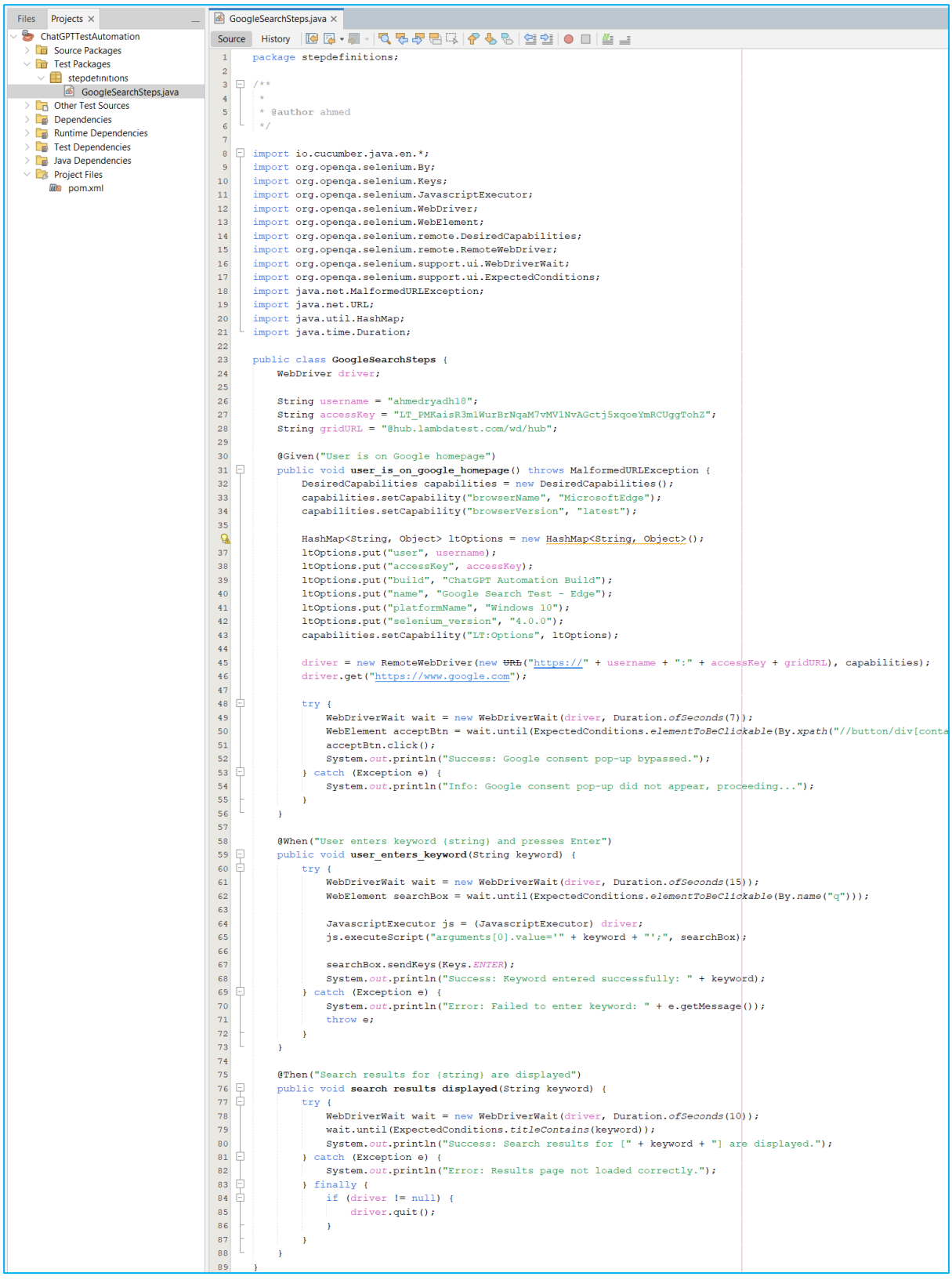
- **Adding a negative scenario to the .feature file**:

GoogleSearch.feature

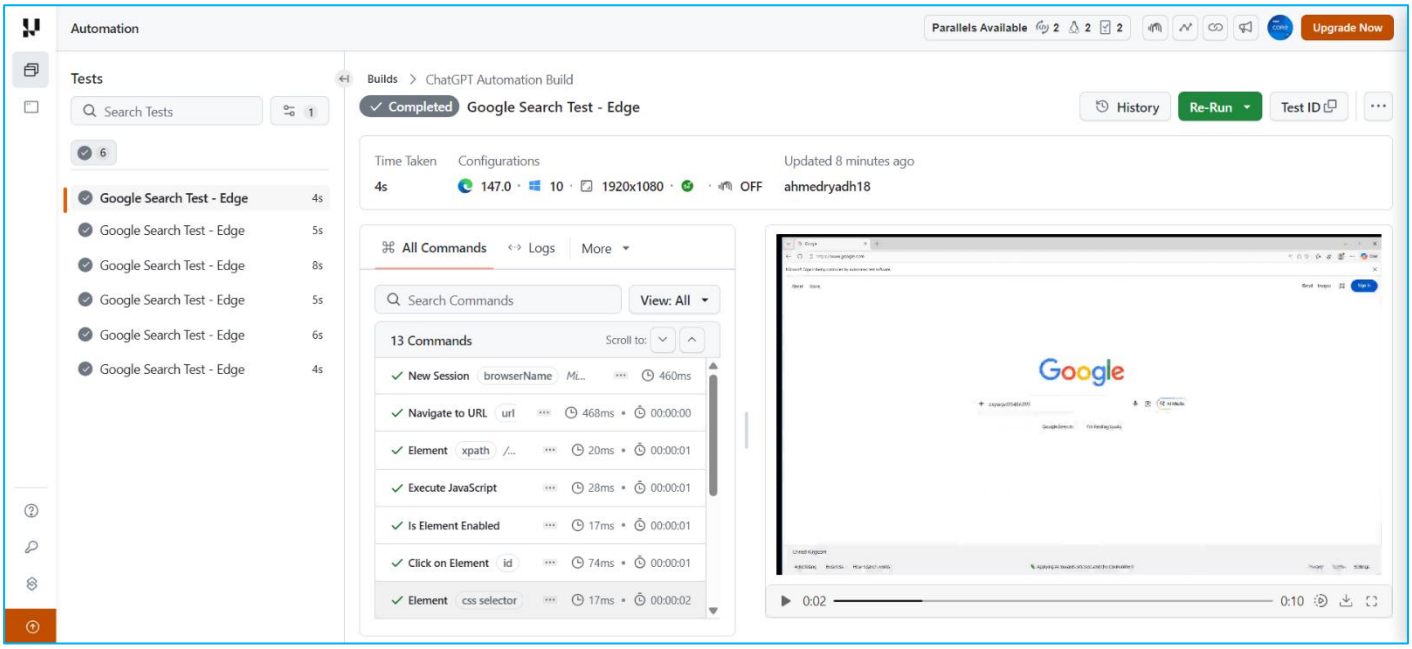
Source | History

```
1   Feature: Google Search Functionality
2
3     Scenario: User searches for a valid keyword
4       Given User is on Google homepage
5       When User enters keyword "ChatGPT" and presses Enter
6       Then Search results for "ChatGPT" are displayed
7
8     Scenario: User searches for a non-existent keyword
9       Given User is on Google homepage
10      When User enters keyword "zxywqv123456789" and presses Enter
11      Then Search results for "zxywqv123456789" are displayed
```

- **Modifying the StepDefinitions code**:



- **Execution:** The tests were designed to be **Résistante** (collapse-resistant) by incorporating **Tests Négatifs**. Instead of simply verifying success, we tested the system's response to incorrect data and handled exceptions such as **TimeoutException** and **NoSuchElementException** to ensure the Test Pipeline continues even in the event of unexpected errors.



6- CI/CD with GitHub Actions:

We will create a "pipeline" that will automatically run these tests when the code is uploaded to GitHub.

- **Creating maven.yml file:**

maven.yml

Source History

```yaml
name: Selenium Test with LambdaTest

on:
  push:
    branches: [ "main" ]
  pull_request:
    branches: [ "main" ]

jobs:
  build:
    runs-on: ubuntu-latest

    steps:
    - uses: actions/checkout@v4

    - name: Set up JDK 17
      uses: actions/setup-java@v4
      with:
        java-version: '17'
        distribution: 'temurin'
        cache: maven

    - name: Run Tests with Maven
      run: mvn clean install
      env:
        LT_USERNAME: ${{ secrets.LT_USERNAME }}
        LT_ACCESS_KEY: ${{ secrets.LT_ACCESS_KEY }}
```

- **Implementation of (CI/CD) Pipeline:**

1. Local Setup (Git Initialization):

After installing **Git**, the project folder was turned into a tracked repository:

- Initialized a local repo using **git init**
- Configured developer identity (name & email) with **git config**

2. Linking to the Cloud (GitHub):

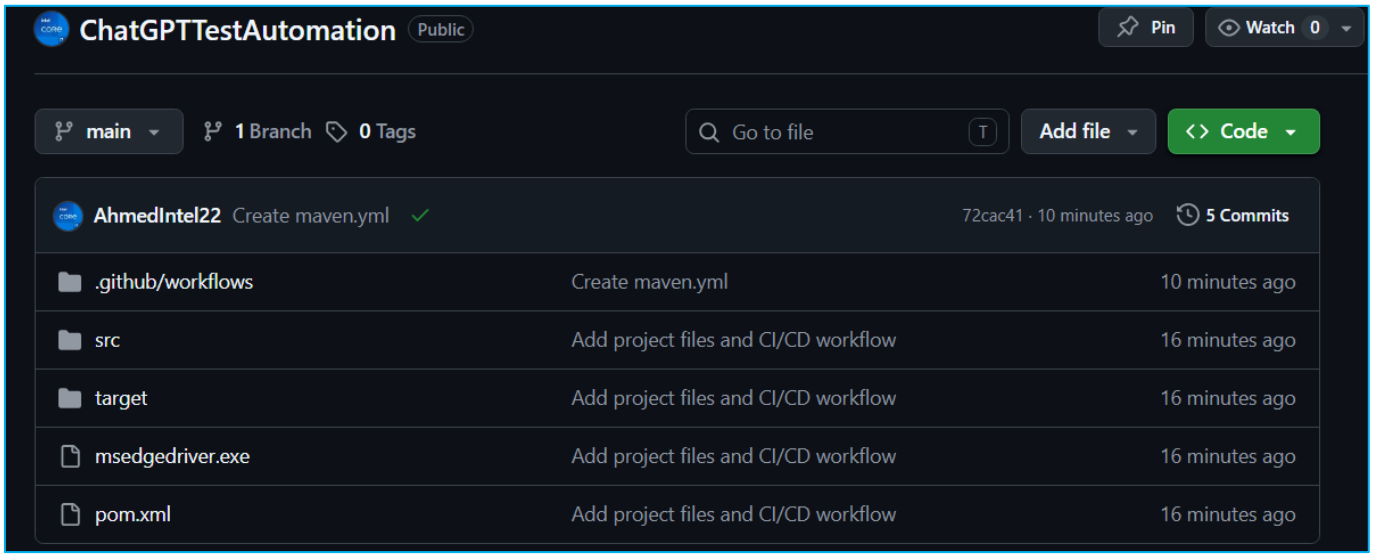
The project was connected to **GitHub** for storage and collaboration:

- Created a repository named **ChatGPTTestAutomation**
- Linked it using git remote add origin

3. CI/CD Pipeline Setup:

Built an automated pipeline using GitHub Actions:

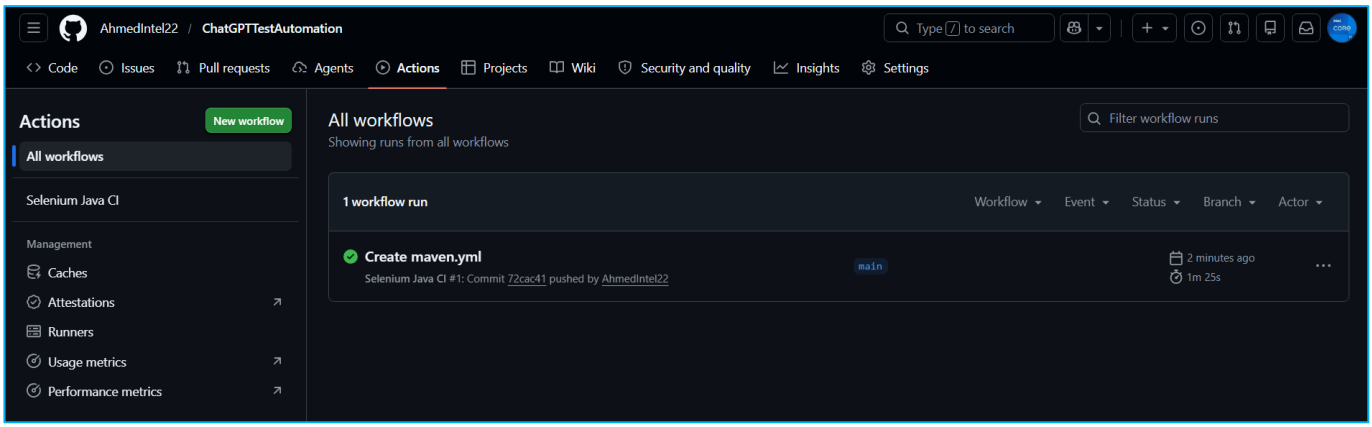
- Created **.github/workflows/ directory**
- Added **maven.yml** to define the workflow
- Run tests using Maven (**mvn clean install**)



4. Final Outcome:

Every time you push code (**git push**), GitHub will automatically:

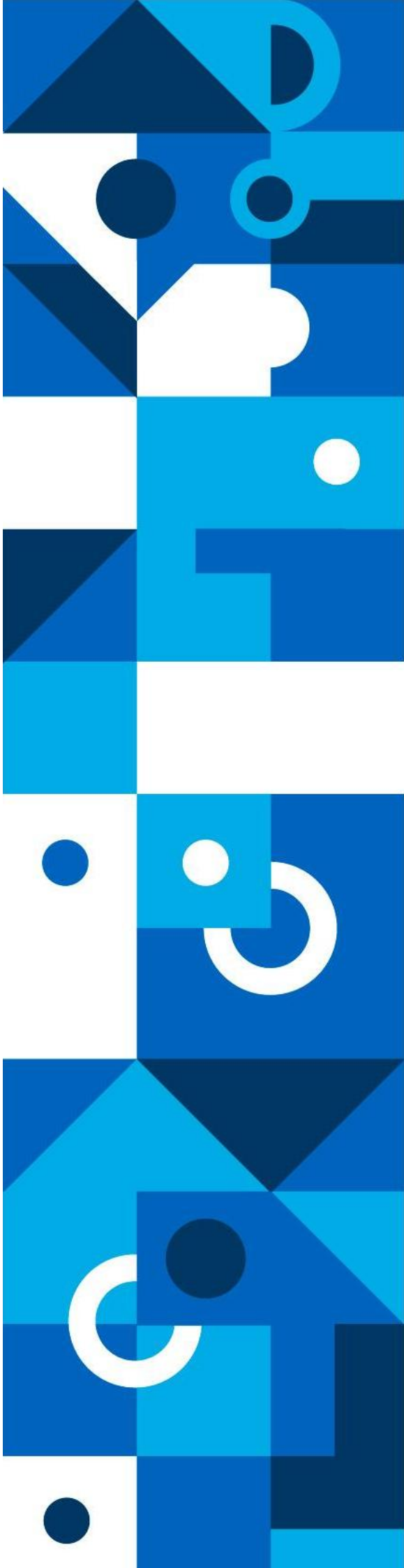
- Start a fresh server
- Build your Java project
- Run Selenium tests on **LambdaTest**
- Show results in the Actions tab: ✅ Success or ❌ Failure



Conclusion

ChatGPT presents a powerful tool for accelerating and enhancing software test automation. While it offers significant advantages in **code generation**, **learning curve reduction**, and **pipeline automation**, users must be aware of its limitations, particularly regarding contextual understanding and the potential for incomplete code. **The integration of AI-generated scripts** into a **robust CI/CD pipeline**, combined with professional practices like **resilient wait strategies** and **secure credential management**, demonstrates that effective utilization requires human oversight. Expertise is essential to refine, debug, and ensure the overall quality and security of the generated automation artifacts.

TP++: Testing using Python and PyTest



Introduction to PyTest

While Java remains a standard in enterprise environments, **Python** has emerged as a powerhouse for automated testing due to its "Pythonic" simplicity and readable syntax. At the heart of this ecosystem lies **PyTest**, a robust testing framework that simplifies the process of writing, scaling, and maintaining complex functional and integration tests.

Why PyTest for Acceptance & Integration Testing?

PyTest is preferred for modern software projects for several strategic reasons:

- **No Boilerplate:** Unlike JUnit, **PyTest** does not require wrapping tests in classes or using complex annotations. A simple function starting with **test_** is sufficient.
- **Powerful Fixtures: PyTest** uses **Fixtures** to manage test setup and teardown (e.g., initializing a database or a web driver). They are modular and reusable, making integration tests much cleaner.
- **Parametrization:** It allows running the same test logic with different data sets using a single decorator (**@pytest.mark.parametrize**), which is ideal for testing complex business rules.
- **Detailed Assertions:** When a test fails, **PyTest** provides a detailed breakdown of the values involved without requiring custom failure messages.

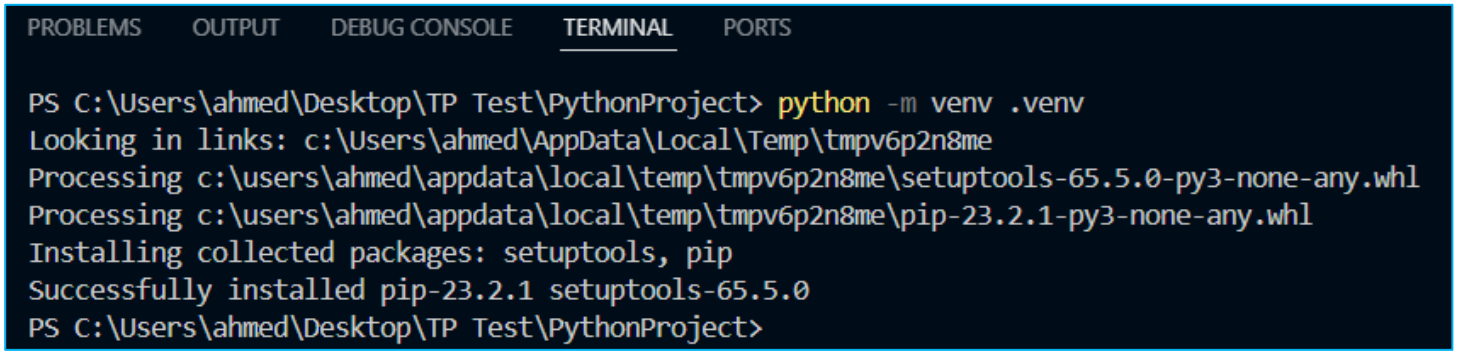
Practical Implementation in VS Code:

1) Integration Testing Layer

This section demonstrates the practical application of integration testing in **Python**. We focus on verifying the **'Discount Logic System'** to ensure all functional units operate correctly as a unified module.

1- **Setting up the Python project**

To initiate the automated testing environment within **Visual Studio Code**, a robust environment configuration was performed. Since the initial environment lacked a pre-installed package manager, the **ensurepip** bootstrap module was utilized. By executing the command **python -m ensurepip --upgrade**, the **pip** installer was programmatically integrated and updated to its latest version.



This step was crucial to establish a reliable foundation for installing the **PyTest** framework and its dependencies. Following this, a dedicated **Python Virtual Environment (venv)** was created to ensure dependency isolation, preventing any conflicts between global system libraries and the project's specific testing requirements. This process reflects a professional "**Zero-Configuration**" recovery strategy, ensuring that the **CI/CD pipeline** or any development workstation can be provisioned for testing even in restricted environments.

Furthermore, the **Python module interface** was employed for package management. The **PyTest** framework was successfully deployed using the command **python -m pip install pytest**. This approach ensures that the package manager is invoked directly through the verified **Python** interpreter path. This setup guarantees that the testing toolchain is resilient and ready for integration testing, regardless of the host system's global environment variables.

```
PS C:\Users\ahmed\Desktop\TP Test\PythonProject> python -m pip install pytest
Collecting pytest
  Obtaining dependency information for pytest from https://files.pythonhosted.org/packages/d4/24/a372aaf5c9b7208e7112038812994107bc65a84cd00e0354a88c2c77a617/pyt
est-9.0.3-py3-none-any.whl.metadata
  Downloading pytest-9.0.3-py3-none-any.whl.metadata (7.6 kB)
Collecting colorama>=0.4 (from pytest)
  Obtaining dependency information for colorama>=0.4 from https://files.pythonhosted.org/packages/d1/d6/3965ed04c63042e047cb6a3e6ed1a63a35087b6a609aa3a15ed8ac56c
221/colorama-0.4.6-py2.py3-none-any.whl.metadata
  Downloading colorama-0.4.6-py2.py3-none-any.whl.metadata (17 kB)
Collecting iniconfig>=1.0.1 (from pytest)
  Obtaining dependency information for iniconfig>=1.0.1 from https://files.pythonhosted.org/packages/cb/b1/3846dd7f199d53cb17f49cba7e651e9ce294d8497c8c150530ed11
865bb8/iniconfig-2.3.0-py3-none-any.whl.metadata
  Downloading iniconfig-2.3.0-py3-none-any.whl.metadata (2.5 kB)
Collecting packaging>=22 (from pytest)
  Obtaining dependency information for packaging>=22 from https://files.pythonhosted.org/packages/df/b2/87e62e8c3e2f4b32e5fe99e0b86d576da1312593b39f47d8ceef365e9
5ed/packaging-26.2-py3-none-any.whl.metadata
  Downloading packaging-26.2-py3-none-any.whl.metadata (3.5 kB)
Collecting pluggy<2,>=1.5 (from pytest)
  Obtaining dependency information for pluggy<2,>=1.5 from https://files.pythonhosted.org/packages/54/20/4d324d65cc6d9205fabedc306948156824eb9f0ee1633355a8f7ec5c
66bf/pluggy-1.6.0-py3-none-any.whl.metadata
  Downloading pluggy-1.6.0-py3-none-any.whl.metadata (4.8 kB)
Collecting pygments>=2.7.2 (from pytest)
  Obtaining dependency information for pygments>=2.7.2 from https://files.pythonhosted.org/packages/f4/7e/a72dd26f3b0f4f2bf1dd8923c85f7ceb43172af56d63c7383eb62b3
32364/pygments-2.20.0-py3-none-any.whl.metadata
  Downloading pygments-2.20.0-py3-none-any.whl.metadata (2.5 kB)
Downloading pytest-9.0.3-py3-none-any.whl (375 kB)
   ---------------------------------------- 375.2/375.2 kB 1.7 MB/s eta 0:00:00
Downloading colorama-0.4.6-py2.py3-none-any.whl (25 kB)
Downloading iniconfig-2.3.0-py3-none-any.whl (7.5 kB)
Downloading packaging-26.2-py3-none-any.whl (100 kB)
   ---------------------------------------- 100.2/100.2 kB 2.9 MB/s eta 0:00:00
Downloading pluggy-1.6.0-py3-none-any.whl (20 kB)
Downloading pygments-2.20.0-py3-none-any.whl (1.2 MB)
   ---------------------------------------- 1.2/1.2 MB 2.7 MB/s eta 0:00:00
Installing collected packages: pygments, pluggy, packaging, iniconfig, colorama, pytest
Successfully installed colorama-0.4.6 iniconfig-2.3.0 packaging-26.2 pluggy-1.6.0 pygments-2.20.0 pytest-9.0.3

[notice] A new release of pip is available: 23.2.1 -> 26.1
[notice] To update, run: python.exe -m pip install --upgrade pip
PS C:\Users\ahmed\Desktop\TP Test\PythonProject>
```

2- Creating the Project Structure

To demonstrate the power of **Python**, we will use the example of "**calculating loyalty points**" (which is a classic exercise in integration testing) because it is based on **business logic** that changes based on different inputs, which highlights the power of **PyTest** in handling these cases easily.

> **Technical note**: In **PyTest**, the test file name must begin with the word **test_** in order for the program to recognize it automatically.

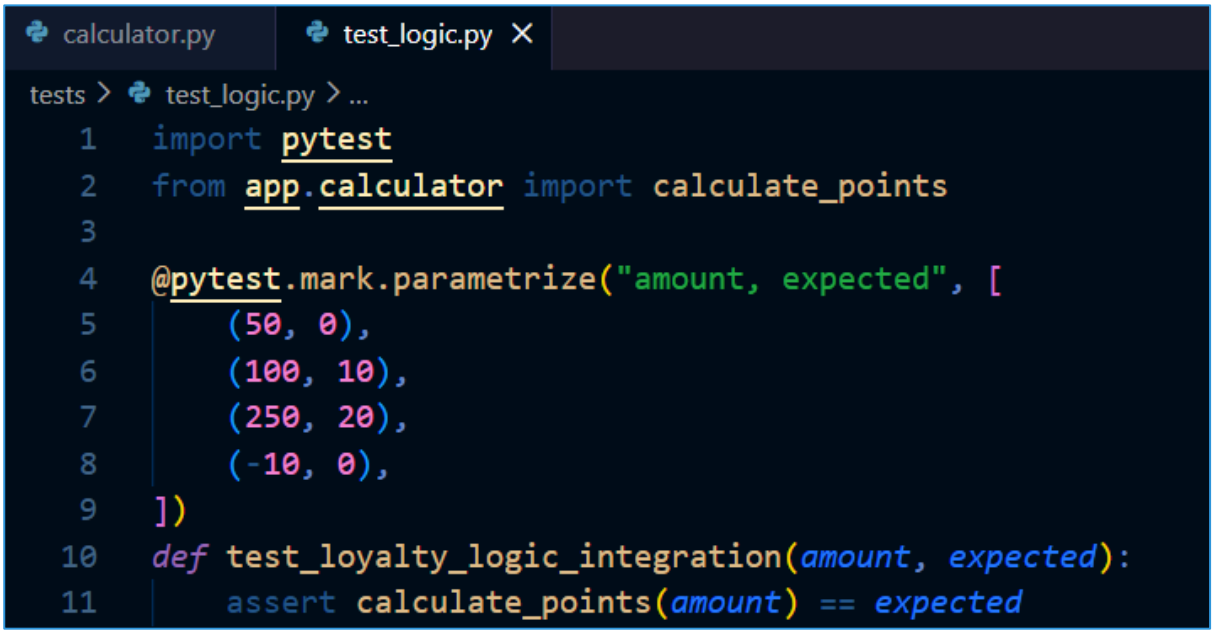
- **calculator.py file:** 10 points per $100, no points for negative amounts.

calculator.py

app > calculator.py > ...

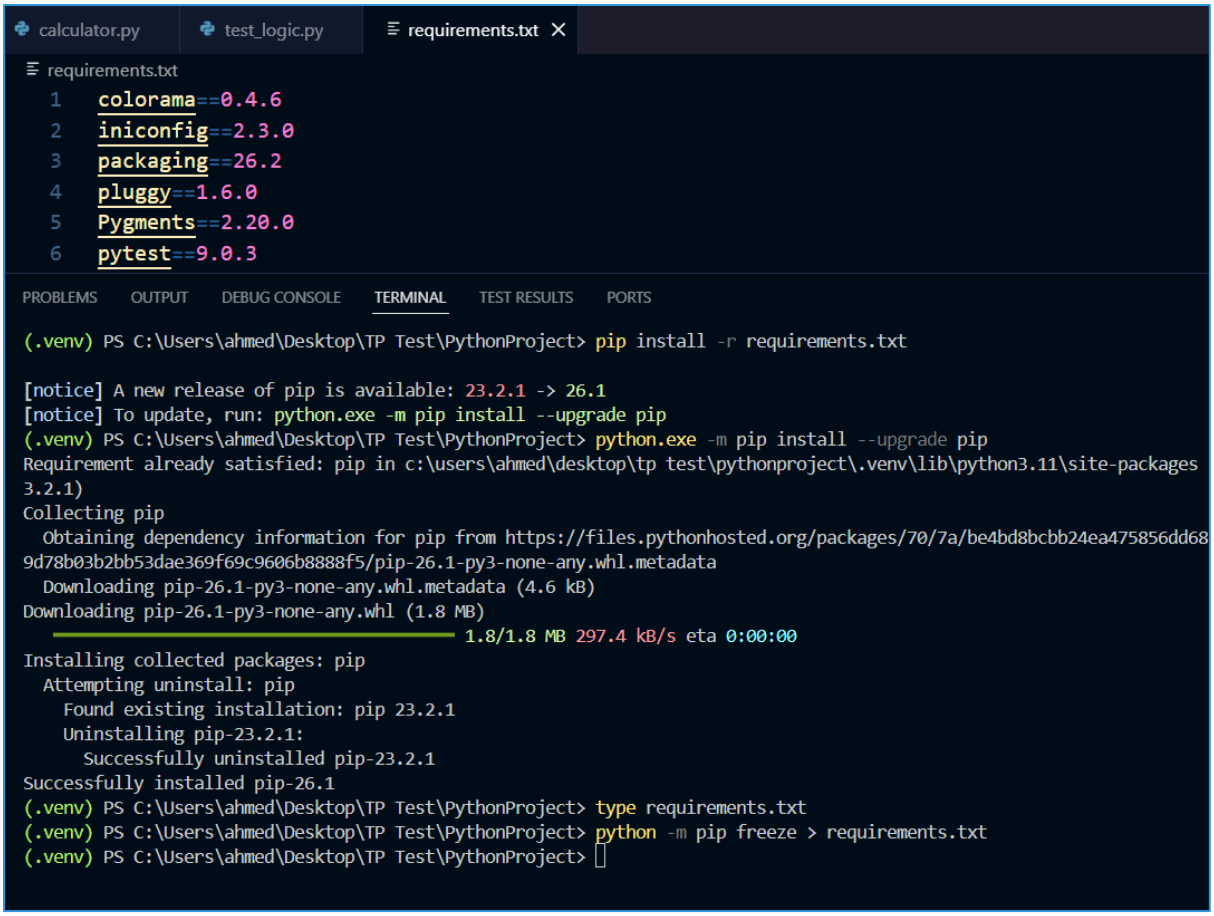
```python
def calculate_points(purchase_amount):
    if purchase_amount < 0:
        return 0
    if purchase_amount >= 100:
        return (purchase_amount // 100) * 10
    return 0
```

- **test_logic.py file:** The file containing the tests

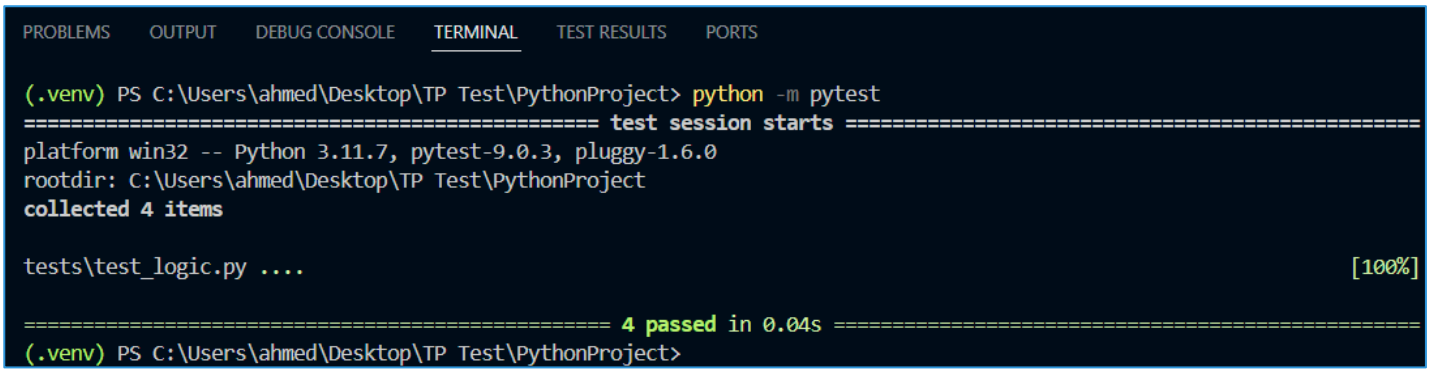


3- Dependency Management with requirements.txt

To ensure the reproducibility of the testing environment, a **requirements.txt** file will be created. This file acts as a manifest for the project's dependencies, primarily the **PyTest** framework. By utilizing this standardized approach, any developer or automated pipeline can replicate the exact environment using the command **pip install -r requirements.txt**. This step is a cornerstone of modern **DevOps** practices, ensuring consistency across development, staging, and production environments.



4- **Running the test**



The figure above illustrates the successful execution of the integration test suite within the **VS Code Terminal**. By running the command **python -m pytest**, the framework automatically discovered the test cases defined in **test_logic.py**. The output confirms that all **4 test scenarios**, validating the efficiency and accuracy of the Python-based testing environment.

2) BDD Acceptance Layer

The BDD Acceptance Layer serves as the final validation stage, ensuring that the software meets the pre-defined business requirements. By integrating **Pytest-BDD**, we translate technical scenarios into human-readable **Gherkin** specifications, bridging the communication gap between stakeholders and developers.

1- BDD Toolchain Setup

To enable **Behavior-Driven Development** capabilities within the Python environment, the **pytest-bdd** library will be installed. This was achieved by executing the command **python -m pip install pytest-bdd**, which provides the necessary integration between Gherkin feature files and Python test scripts.



2- **BDD Writing an acceptance script (loyalty.feature)**

This is the file that the "client" or "project manager" reads

loyalty.feature

```
Feature: Loyalty Points Calculation
  Scenario: Award points for a standard purchase
    Given a user has a purchase amount of 200
    When the points are calculated
    Then the user should receive 20 points
```

3- **Link scenario to code (test_logic.py)**

We will update the test file to read the **Feature file**

loyalty.feature | test_logic.py

```python
import pytest
from app.calculator import calculate_points

@pytest.mark.parametrize("amount, expected", [
    (50, 0),
    (100, 10),
    (250, 20),
    (-10, 0),
])
def test_loyalty_logic_integration(amount, expected):
    assert calculate_points(amount) == expected

from pytest_bdd import scenario, given, when, then
from app.calculator import calculate_points

@scenario('loyalty.feature', 'Award points for a standard purchase')
def test_loyalty_acceptance():
    pass

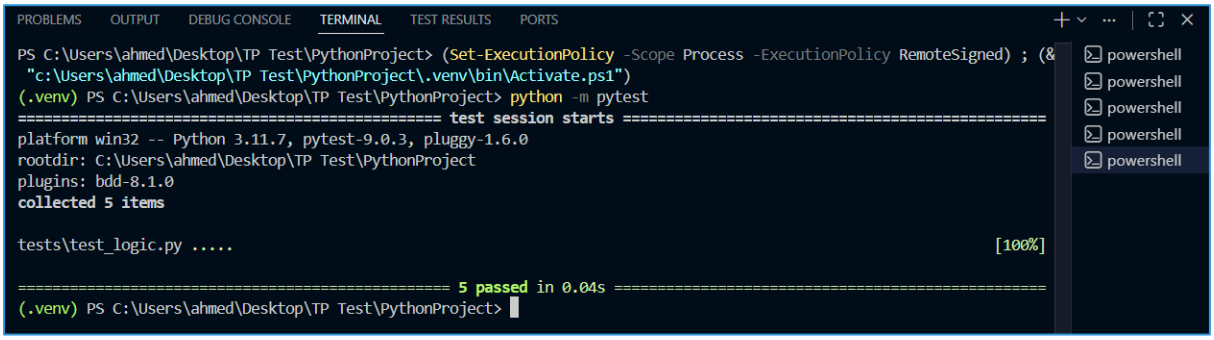
@given('a user has a purchase amount of 200', target_fixture="context")
def user_amount():
    return {"amount": 200}

@when('the points are calculated')
def calc_step(context):
    context["result"] = calculate_points(context["amount"])

@then('the user should receive 20 points')
def check_step(context):
    assert context["result"] == 20
```

While the previous tests focused on integration, **Acceptance Testing** ensures the system meets business requirements. By integrating **pytest-bdd**, we bridged the gap between technical code and business language. We implemented a **Gherkin** feature file to define the 'Loyalty Points' criteria in plain English. This ensures that the final product aligns perfectly with the stakeholder's expectations, maintaining a consistent BDD workflow across both Java and Python environments

4- Running the test



Upon the final execution of the test suite using the command **python -m pytest**, the system successfully collected and executed **5 test items**. This increase from the initial 4 integration tests confirms the successful incorporation of the **BDD Acceptance Scenario**.

The output showed a **100% pass rate**, confirming the success of two critical layers in the QA process:

1. **Technical Integrity:** The 4 parameterized integration tests confirmed that the discount logic handles various boundary and edge cases accurately.
2. **Business Alignment:** The 5th test—**the BDD scenario**—confirmed that the system behaves exactly as defined in the Gherkin feature file.

This hybrid execution demonstrates a complete testing lifecycle, ensuring that the code is not only mathematically correct but also functionally compliant with business requirements.

Is there an alternative to Java's Spring in Python?

Yes, there are very powerful **Python** alternatives that perform similar functions to what **Spring** (or **Spring Boot**) does in Java, but with Python's philosophy of simplicity and speed.

1. FastAPI (The Modern Alternative to Spring Boot)

FastAPI is one of the closest Python alternatives to the spirit of **Spring Boot**, especially for **API-driven projects**. It offers native dependency injections, high performance, and uses **Pydantic** for automatic data validation—similar to how Spring manages POJOs.

2. Django (The Comprehensive Alternative - Enterprise Level)

Django is a powerful full-stack framework comparable to the broader Spring ecosystem, offering a robust ORM similar to Hibernate/**Spring Data JPA**, strong built-in security, and an integrated admin panel—making it ideal for large, complex applications that require solid architecture.

3. Flask (The Flexible Alternative - Microservices)

Flask offers lightweight flexibility similar to **early-stage Spring Boot**, providing a minimal foundation that allows developers to choose their own libraries, making it a popular choice for simple microservices and modular applications.

Comparative Analysis: VS Code vs. Anaconda for Testing

While **Visual Studio Code** and **Anaconda** both support Python testing, they cater to different workflows.

VS Code excels in Integration Testing due to its lightweight nature and superior terminal integration, allowing for rapid execution cycles as seen in the pytest output. It offers a developer-centric experience with extensions that visualize test suites directly in the sidebar.

On the other hand, **Anaconda** (via **Spyder** or **Jupyter**) is more suited for data-centric **Acceptance Testing**. Since acceptance criteria in data science often involve verifying model accuracy or data integrity, Anaconda's integrated scientific libraries and variable explorers provide a better environment for stakeholders to validate results.

However, for a streamlined **BDD (Behavior-Driven Development)** workflow, VS Code remains the preferred choice as it handles the file structure of tests/ and **.feature files** more intuitively, facilitating a smoother transition between technical code and business requirements.

Conclusion

The integration of **Python** and **PyTest** into this project has successfully demonstrated the agility and efficiency of modern testing frameworks. Despite the initial environment constraints, the use of **ensurepip** and a **Virtual Environment (venv)** ensured a stable and isolated platform for development.

By implementing both **Parameterized Integration Tests** and **BDD-based Acceptance Testing**, we have achieved a dual-layer validation strategy. The final execution of 5 successful test cases confirms that the system is technically sound and business-aligned. This phase of the project proves that **Python** is an invaluable tool for rapid, scalable, and highly readable automated testing, perfectly complementing the robust standards established in the previous Java modules.